
Master Degree in Computer Science
A.Y. 2024- 2025

HUMAN LANGUAGE TECHNOLOGIES

RECURRENT NEURAL NETWORKS

Teachers

A. Carta, A. Ceni, A. Cossu, C. Gallicchio

CREDITS
LUCIA C. PASSARO
lucia.passaro@unipi.it

OUTLINE

- Short recap (LM, N-gram, FF neural nets)
- Recurrent Neural Networks (RNNs)
- RNNs as Language Models
- RNNs for other NLP tasks
- Stacked and bi-directional RNNs
- Gated neural network architectures: LSTM
- Encoder-decoder
- Attention

WHAT TO STUDY



Daniel Jurafsky and James H. Martin. 2025.
[Speech and Language Processing: An
Introduction to Natural Language Processing,
Computational Linguistics, and Speech
Recognition with Language Models](https://web.stanford.edu/~jurafsky/slp3/), 3rd
edition.

<https://web.stanford.edu/~jurafsky/slp3/>

Chapter 8



SHORT RECAP

LANGUAGE MODELING: RECAP

A **language model** (LM) is a machine learning model that **predicts upcoming words**.

LANGUAGE MODELING: RECAP

A **language model** (LM) is a machine learning model that **predicts upcoming words**.

- Assign a probability to an entire sequence
- Assign a probability distribution over possible next words

LANGUAGE MODELING: RECAP

- A Language Model **assigns a probability to each sentence** in a language:

$$P(w_1, w_2, w_3, w_4, w_5 \dots w_n) = P(W)$$

= the probability of a sequence

- Alternatively, we want to compute

$$P(w_n | w_1, \dots, w_{n-1})$$

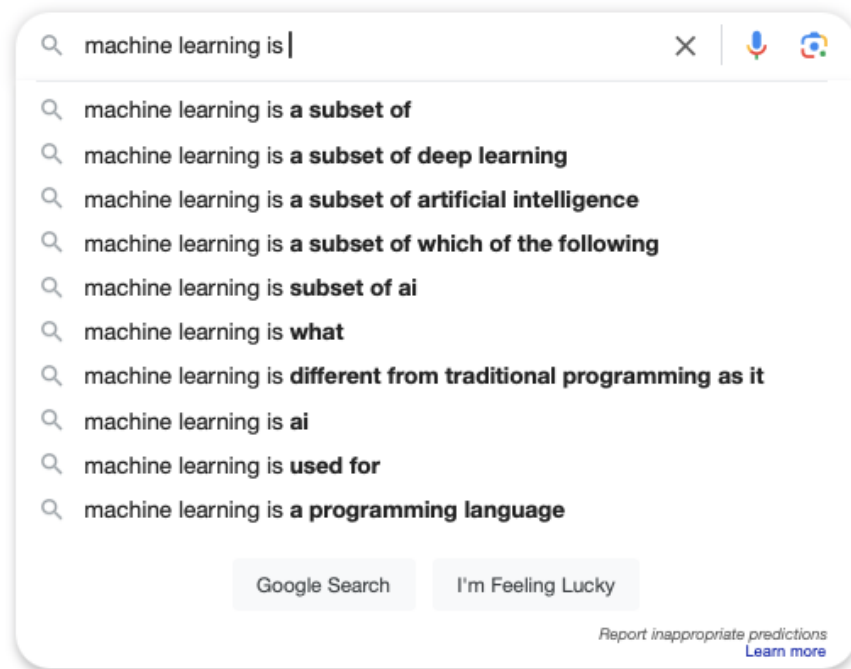
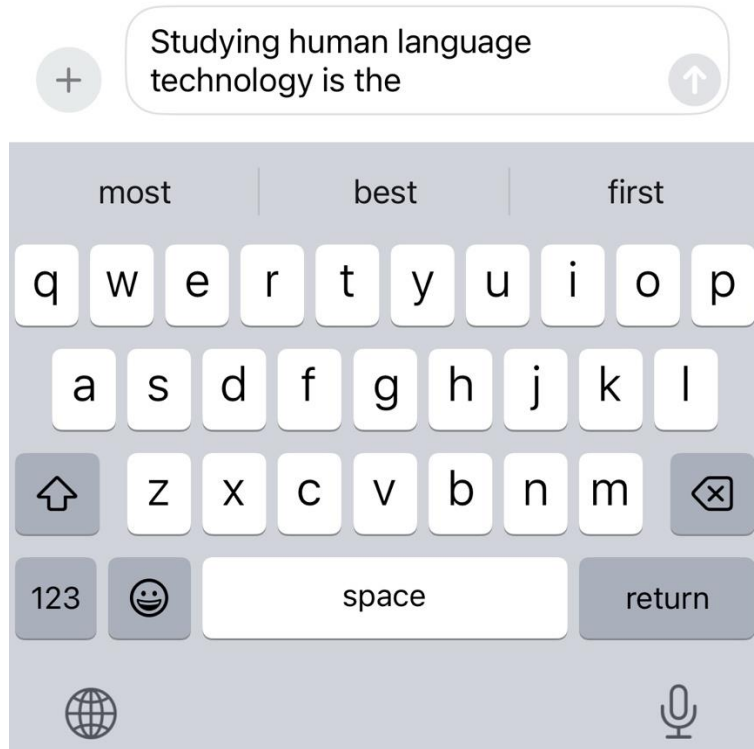
= the probability of a word given some previous words

- The model that computes

$$P(W) \text{ or}$$

$$P(w_n | w_1, w_2 \dots w_{n-1})$$

is called the **language model**



MARKOV ASSUMPTION (N-GRAM LANGUAGE MODEL)

Markov assumption: x_n depends only on the preceding $N-1$ words

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-N+1}^{t-1})$$

Maximum
likelihood
estimation

MLE:

$$P(w_t | w_{t-N+1}, \dots, w_{t-1}) = \frac{\text{count}(w_{t-N+1}, \dots, w_{t-1}, w_t)}{\text{count}(w_{t-N+1}, \dots, w_{t-1})}$$

PROBLEMS WITH N-GRAM MODELS

Sparsity

Since the number of possible N-grams grows exponentially with the size of the vocabulary, many word combinations may not appear in the training corpus.

- OOV words (solution: **smoothing**, add 1 to counts)

Shave off a bit of probability mass from some more frequent events and give it to unseen words.

- Prefix never seen (solution: **backoff**, drop some words from prefix)

If the n-gram has zero counts, we approximate it by backing off to the (n-1)-gram

PROBLEMS WITH N-GRAM MODELS

Storage

- An n-gram model must store the counts of all n-grams seen in the corpus, and the number of these grows as $|V|^n$
- If you increase n or use a larger corpus, the model quickly becomes unmanageable in terms of memory.

PROBLEMS WITH N-GRAM MODELS

Effects of increasing n

- Increasing n worsens the sparsity problem
 - the larger n is, the less frequently certain word sequences will appear in the training corpus.
- In practice, to avoid these problems, n usually does not exceed a value of 5.

FIXED-WINDOW NEURAL LANGUAGE MODEL

~~as the proctor started the clock~~ the students opened their _____

discard

fixed window

Output distribution:

$$\hat{y} = \text{softmax}(U\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

Hidden layer :

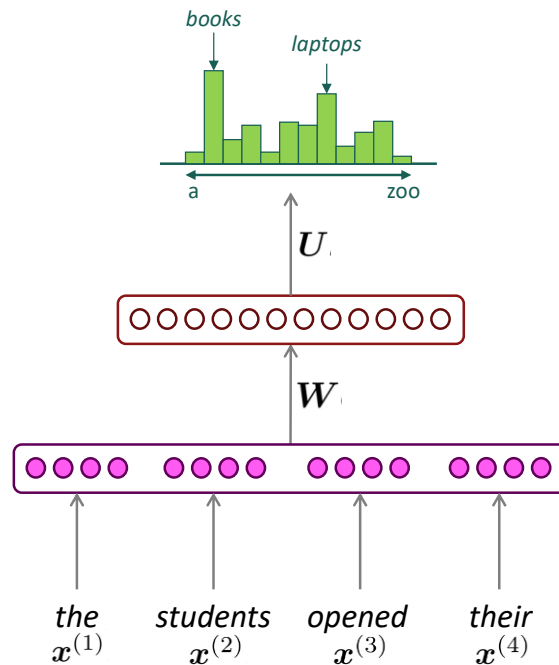
$$\mathbf{h} = f(\mathbf{W}\mathbf{e} + \mathbf{b}_1)$$

Concatenated word embeddings:

$$\mathbf{e} = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

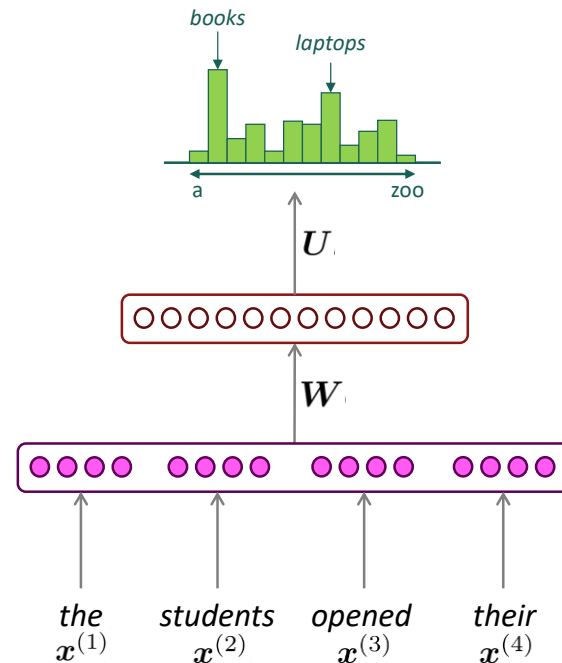
Words, one-hot encoded:

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$



NEURAL LANGUAGE MODEL

- **Improvements** over n -gram LM:
 - No sparsity problem
 - Don't need to store all observed n -grams
- Remaining **problems**:
 - Fixed window is **too small**
 - Enlarging window enlarges W
 - Window can never be large enough!
 - $x(0)$ and $x(1)$ are multiplied by completely different weights in W . No symmetry in how the inputs are processed.
- We need a neural architecture that can process **any length input**



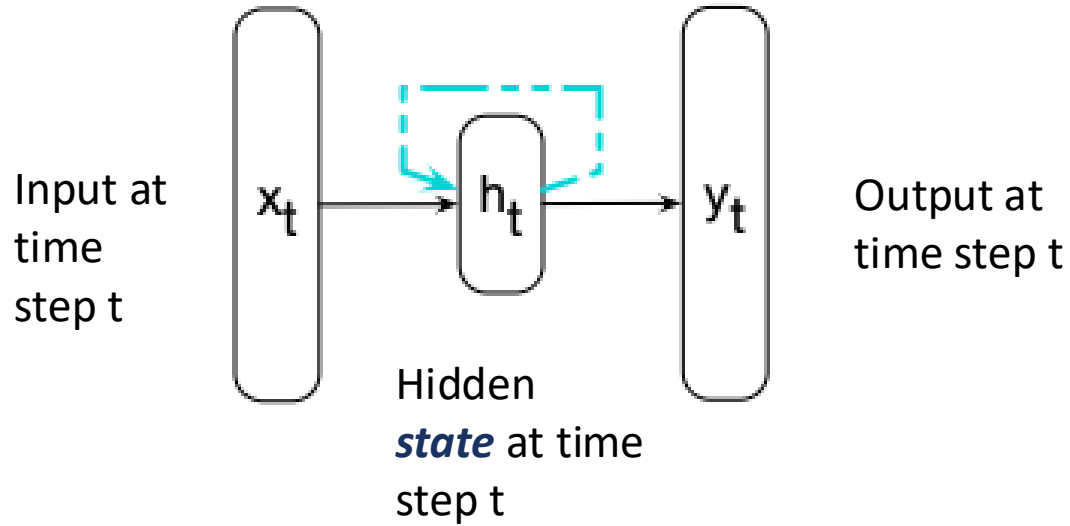
RECURRENT NEURAL NETWORKS (RNNS)

Unlike models seen so far, Recurrent Neural Networks (RNNs) have a mechanism that deals directly with the sequential nature of language

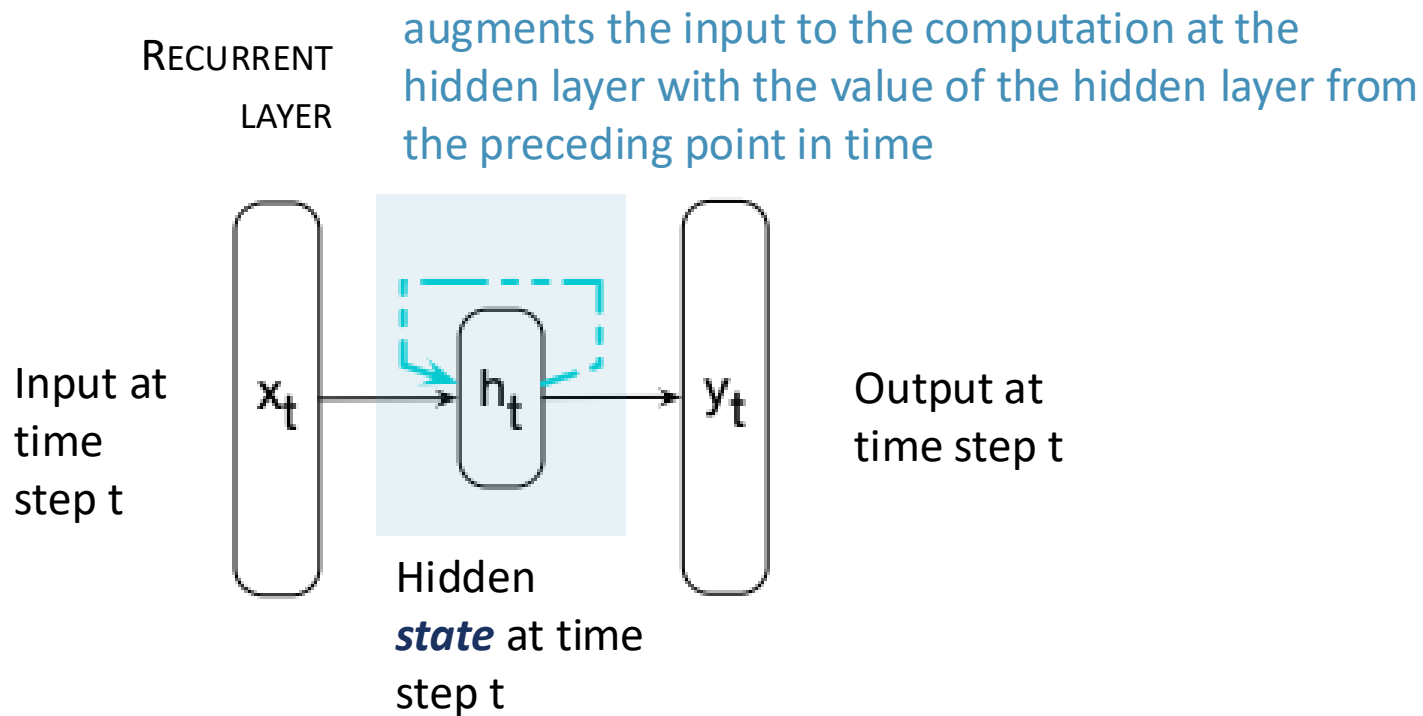
Handle the temporal nature of language without the use of arbitrary fixed-sized windows.

The RNN approach offers a new way to represent the prior context, in its recurrent connections, allowing the model's decision to depend on information from hundreds of words in the past.

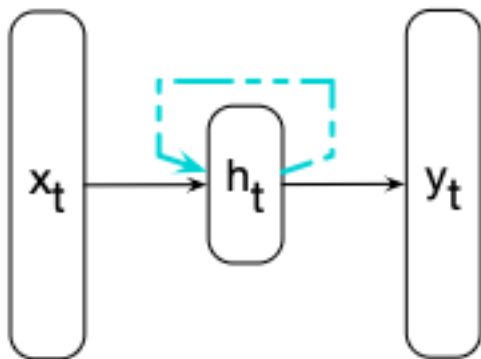
SIMPLE RECURRENT NETWORK – VANILLA RNN



VANILLA RNN

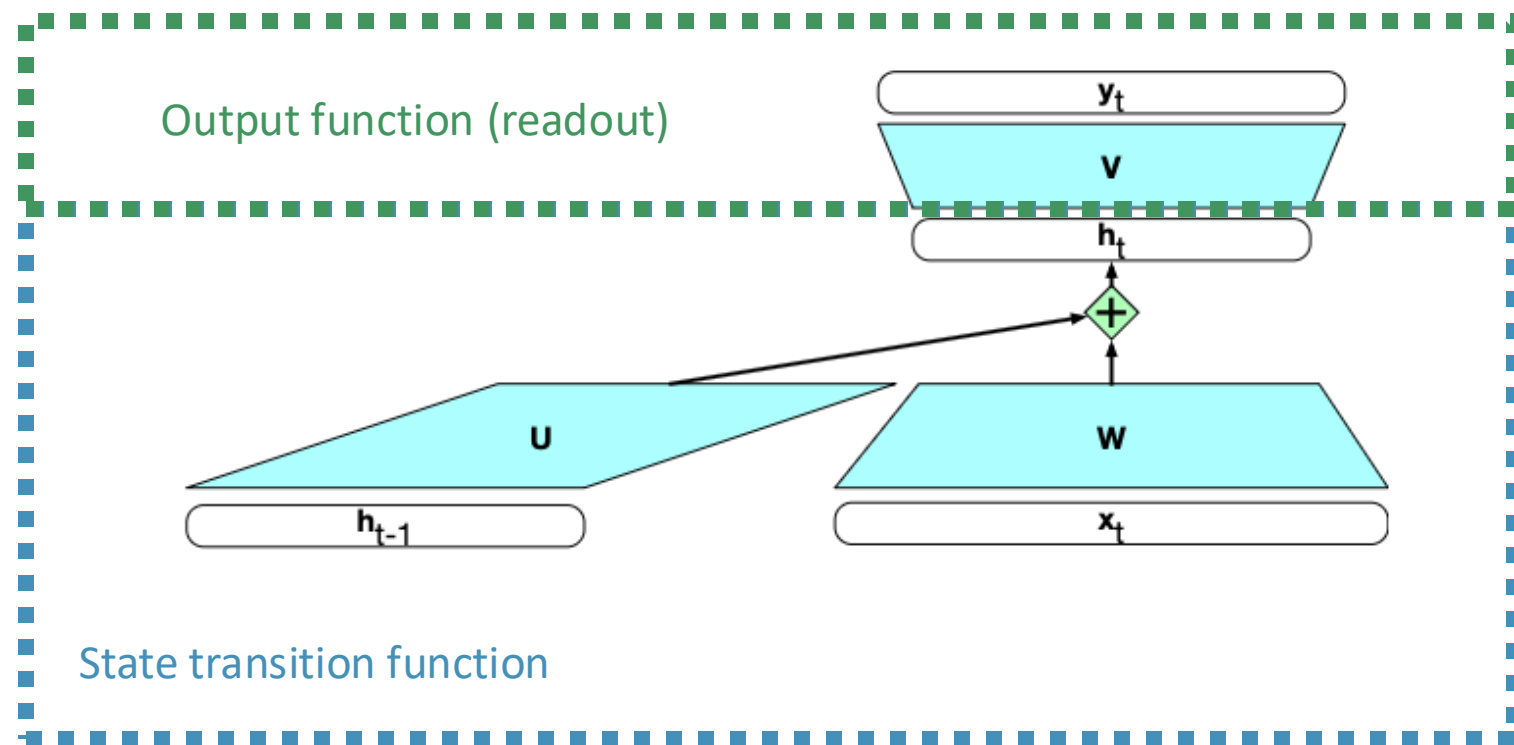


VANILLA RNN

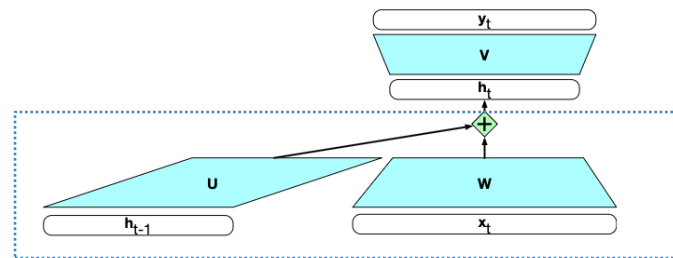


- The hidden layer from the previous time step provides a form of **memory**, or context.
- This is called the **state** of the recurrent layer.
- The state **encodes earlier processing** and informs the decisions to be made at later points in time.
- *Does not impose a fixed-length limit on the prior context.*

VANILLA RNN - INFERENCE



VANILLA RNN - INFERENCE



$$\underset{\text{state at time } t}{\mathbf{h}_t} = g\left(\underset{\text{state at time } t-1}{\mathbf{U}\mathbf{h}_{t-1}} + \underset{\text{input at time } t}{\mathbf{W}\mathbf{x}_t}\right)$$

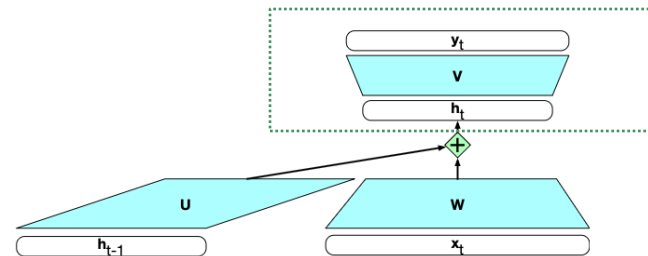
recurrent weight matrix

input weight matrix

$$\mathbf{h}_t \in \mathbb{R}^{d_h} \quad \mathbf{x}_t \in \mathbb{R}^{d_{in}}$$

$$\mathbf{U} \in \mathbb{R}^{d_h \times d_h} \quad \mathbf{W} \in \mathbb{R}^{d_h \times d_{in}}$$

VANILLA RNN - INFERENCE



output at time t $\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$ state at time t

output weight matrix

$$\mathbf{h}_t \in \mathbb{R}^{d_h} \quad \mathbf{y}_t \in \mathbb{R}^{d_{out}} \quad \mathbf{V} \in \mathbb{R}^{d_{out} \times d_h}$$

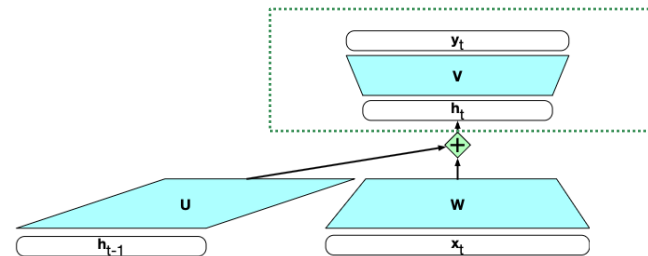
VANILLA RNN - INFERENCE

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

output at time t output weight matrix state at time t

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

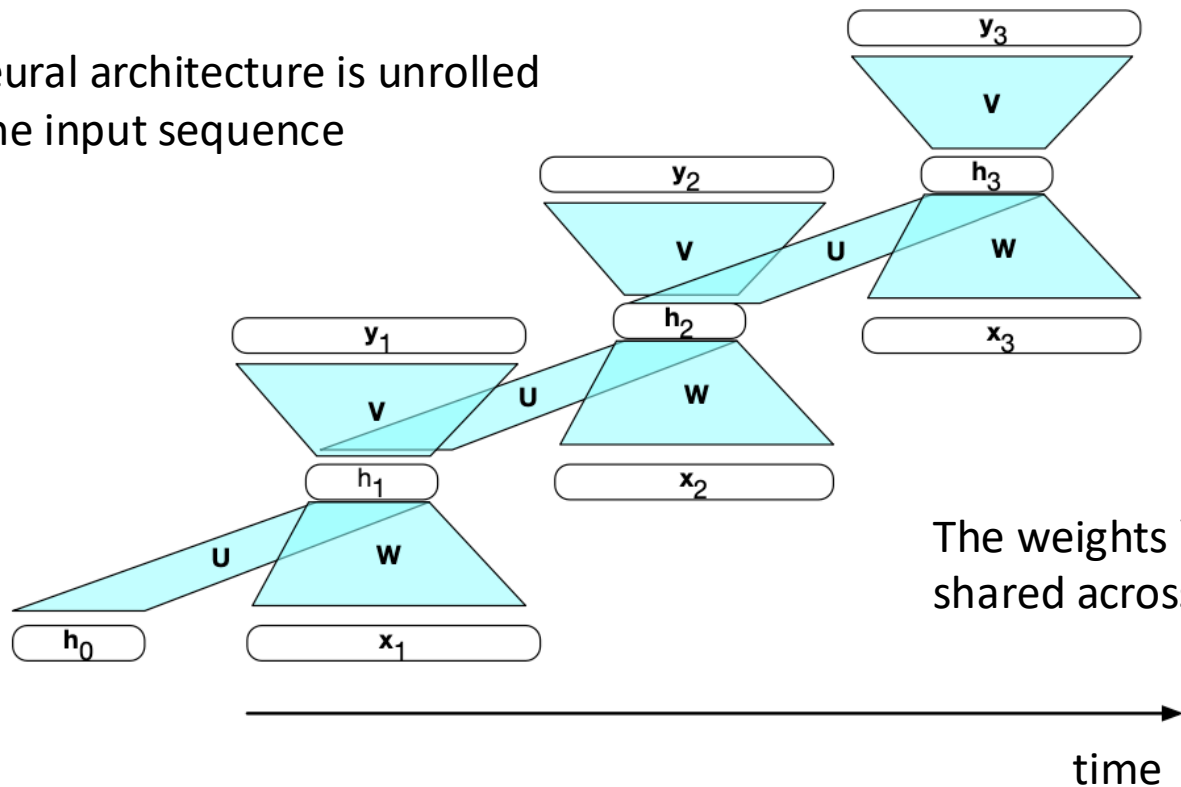
$$\mathbf{h}_t \in \mathbb{R}^{d_h} \quad \mathbf{y}_t \in \mathbb{R}^{d_{out}} \quad \mathbf{V} \in \mathbb{R}^{d_{out} \times d_h}$$



In case of sequence classification we use a softmax activation function for the output neurons

VANILLA RNN UNROLLED IN TIME

The neural architecture is unrolled over the input sequence

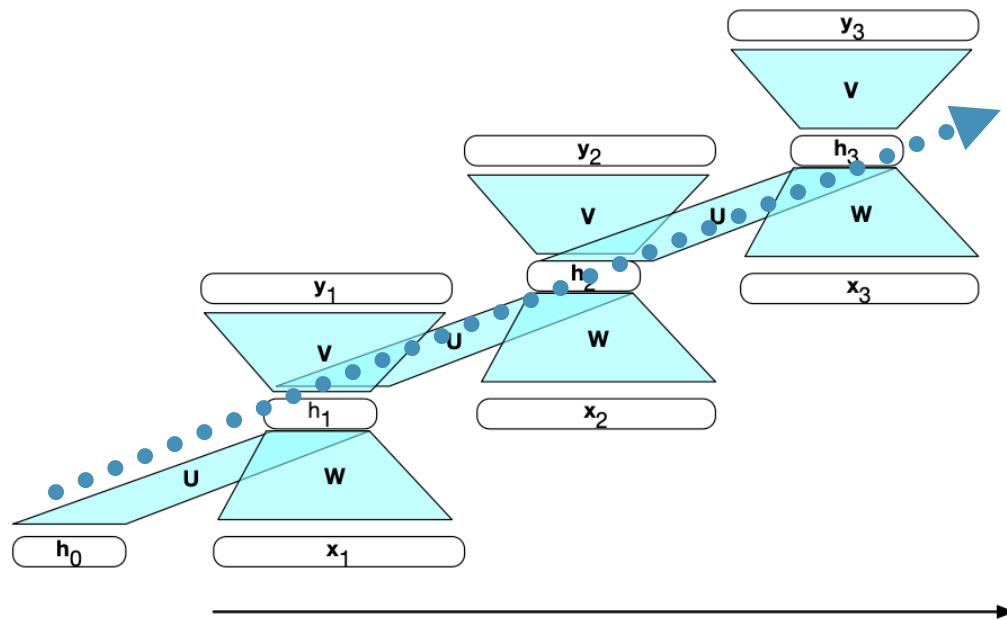


VANILLA RNN - TRAINING

- We still use a training set, a loss function and backpropagation to obtain gradients and adjust the weights in the RNN.
 - Now, we have 3 sets of weights: **U**, **V**, **W**
- Backpropagation through time (BPTT)

Werbos, P. J. 1990. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560.

VANILLA RNN - TRAINING

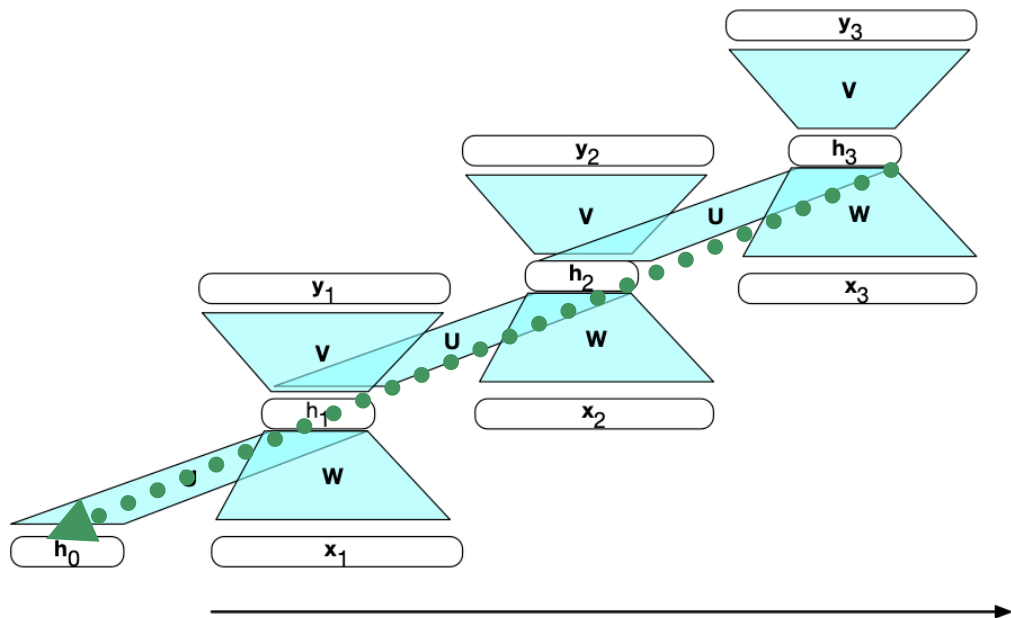


Forward pass:

The computational graph is traversed following the temporal order.

Compute states h_t , outputs y_t , and loss values.

VANILLA RNN - TRAINING



Backward pass:

Propagate gradients in reverse from the last time-step all the way to the first one.

The computational graph is traversed in reversed order.

RNNS AS LANGUAGE MODELS

Recall: language models predict the **next word** in a sequence given some preceding context

$$P(\textit{fish}|\textit{Thank you for all the})$$

Recall: language models predict the **next word** in a sequence given some preceding **context**

$$P(\textit{fish}|\textit{Thank you for all the})$$

RNNs as language models: process the input sequence one word at a time, attempting to **predict the next word** from the **current word** and the **previous hidden state**.

- The hidden state can in principle represent information about all the preceding words
- The language model uses h_{t-1} as a representation of the past context.

FORWARD INFERENCE IN AN RNN LANGUAGE MODEL

- Start with an input sequence $\mathbf{X} = [\mathbf{x}_1; \dots; \mathbf{x}_t; \dots; \mathbf{x}_N]$
- Compute the embeddings of each word

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t$$

- Update the state of the recurrent layer over the entire sequence

$$\mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t)$$

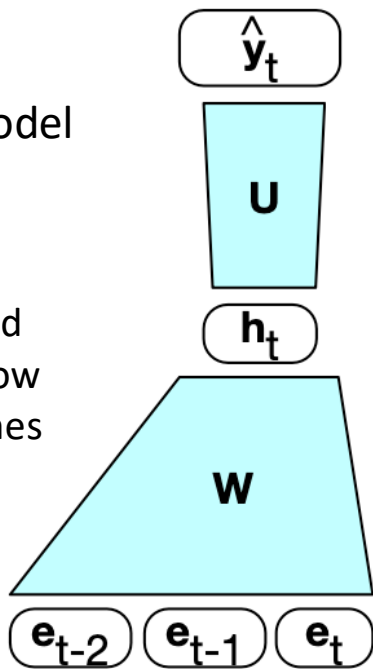
- Compute the output using a softmax act. function

$$\hat{\mathbf{y}}_t = \textit{softmax}(\mathbf{V} \mathbf{h}_t)$$

FORWARD INFERENCE IN AN RNN LANGUAGE MODEL

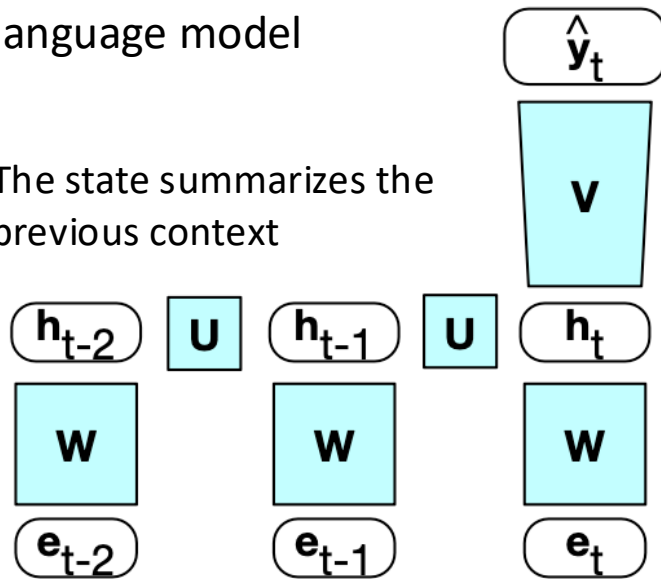
FF neural
language model

There is a fixed
context window
that determines
the context



RNN neural
language model

The state summarizes the
previous context



TRAINING AN RNN LANGUAGE MODEL

Use self-supervision.

- Take a corpus of text as training material
- At each time step t , ask the model to predict the next word

$$w_{t+1}$$

The model is trained to minimize the error in predicting the true next word in the training sequence.

TRAINING AN RNN LANGUAGE MODEL

Use **cross-entropy loss**.

- Measure the difference between a predicted probability distribution and the correct distribution
- For language modeling, this is determined by the probability assigned by the model to the correct next word

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

TRAINING AN RNN LANGUAGE MODEL

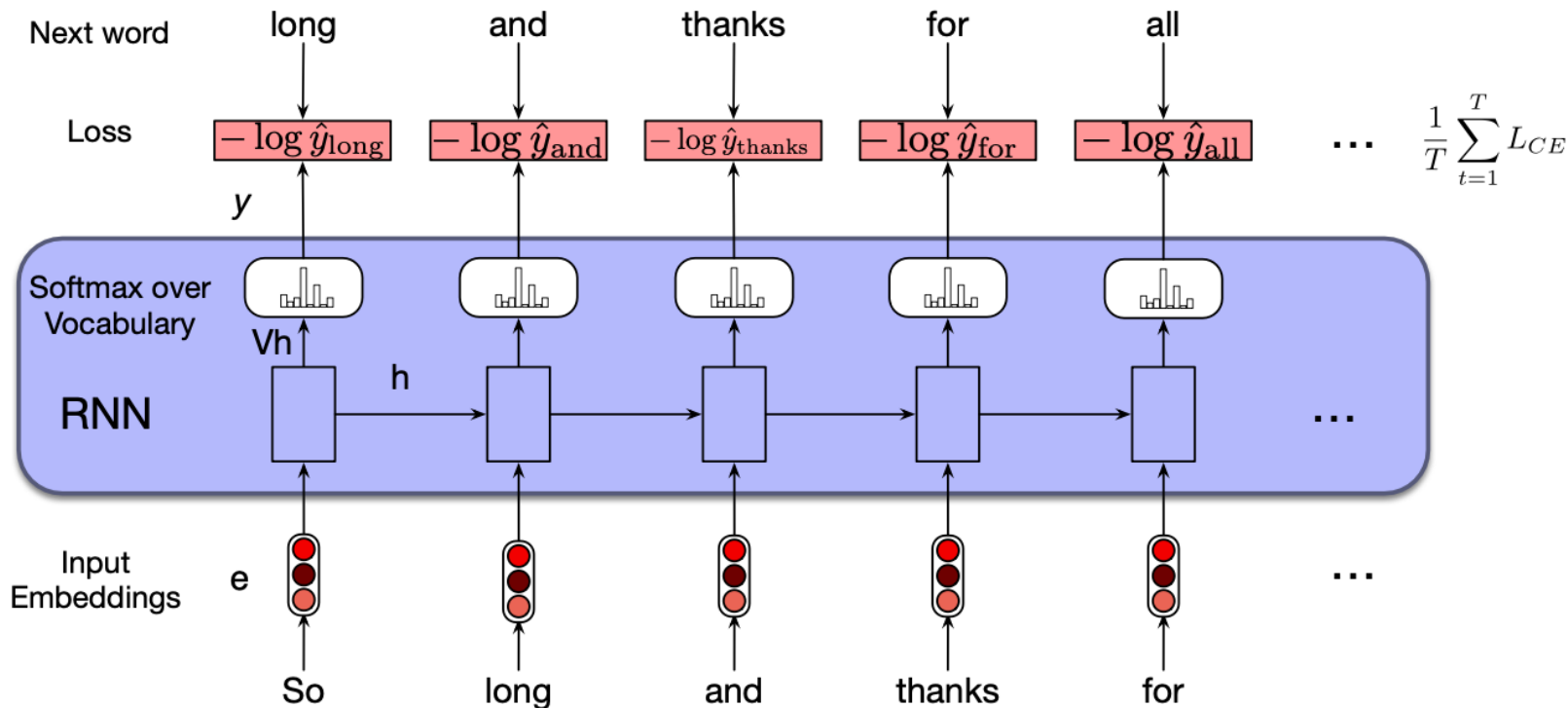
Use **teacher forcing**.

For each position t of the input:

- Take in input the correct word w_t and the state h_{t-1} (that encodes $w_{1:t-1}$), and compute a probability distribution over possible next words
- Compute the model's loss for the next token w_{t+1}

The input at time t is not the output of the model at time $t - 1$, but the correct word from the training set

TRAINING AN RNN LANGUAGE MODEL



WEIGHT TYING

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t)$$

$$\hat{\mathbf{y}}_t = \textit{softmax}(\mathbf{V} \mathbf{h}_t)$$

$$\mathbf{x}_t \in \mathbb{R}^{|V|}$$

WEIGHT TYING

$$\mathbf{x}_t \in \mathbb{R}^{|V|}$$

$$\mathbf{e}_t, \mathbf{h}_t \in \mathbb{R}^d$$

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t)$$

$$\hat{\mathbf{y}}_t = \textit{softmax}(\mathbf{V} \mathbf{h}_t)$$

The embedding dimension and the hidden dimension are the same (the model dimension)

WEIGHT TYING

$$\begin{aligned}\mathbf{e}_t &= \mathbf{E} \mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t) \\ \hat{\mathbf{y}}_t &= \textit{softmax}(\mathbf{V} \mathbf{h}_t)\end{aligned}$$

$$\mathbf{x}_t \in \mathbb{R}^{|V|}$$

$$\mathbf{e}_t, \mathbf{h}_t \in \mathbb{R}^d$$

$$\mathbf{W} \in \mathbb{R}^{d \times d}$$

$$\mathbf{U} \in \mathbb{R}^{d \times d}$$

The embedding dimension and the hidden dimension are the same (the model dimension)

WEIGHT TYING

$$\begin{aligned}\mathbf{e}_t &= \mathbf{E} \mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t) \\ \hat{\mathbf{y}}_t &= \textit{softmax}(\mathbf{V} \mathbf{h}_t)\end{aligned}$$

$$\mathbf{x}_t \in \mathbb{R}^{|V|}$$

$$\mathbf{e}_t, \mathbf{h}_t \in \mathbb{R}^d$$

$$\mathbf{W} \in \mathbb{R}^{d \times d}$$

$$\mathbf{U} \in \mathbb{R}^{d \times d}$$

$$\mathbf{E} \in \mathbb{R}^{d \times |V|}$$

The columns of $\mathbf{E}$ represent the embeddings for each word in the vocabulary

$$\mathbf{E} = \begin{bmatrix} | & | & \dots & | \\ \mathbf{e}_{w_1} & \mathbf{e}_{w_2} & \dots & \mathbf{e}_{w_{|V|}} \\ | & | & \dots & | \end{bmatrix}$$

WEIGHT TYING

$$\begin{aligned}\mathbf{e}_t &= \mathbf{E} \mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t) \\ \hat{\mathbf{y}}_t &= \textit{softmax}(\mathbf{V} \mathbf{h}_t)\end{aligned}$$

$$\mathbf{x}_t \in \mathbb{R}^{|V|}$$

$$\mathbf{e}_t, \mathbf{h}_t \in \mathbb{R}^d$$

$$\mathbf{W} \in \mathbb{R}^{d \times d}$$

$$\mathbf{U} \in \mathbb{R}^{d \times d}$$

$$\mathbf{E} \in \mathbb{R}^{d \times |V|}$$

$$\mathbf{V} \in \mathbb{R}^{|V| \times d}$$

$\mathbf{V}$ provides a way to score the likelihood of each word in the vocabulary (based on each $\mathbf{h}_t$)

- The rows of $\mathbf{V}$ are shaped as the **transpose** of the columns of $\mathbf{E}$.
- $\mathbf{V}$ provides **a second set of learned embeddings**

WEIGHT TYING

$$\begin{aligned}\mathbf{e}_t &= \mathbf{E} \mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t) \\ \hat{\mathbf{y}}_t &= \text{softmax}(\mathbf{E}^T \mathbf{h}_t)\end{aligned}$$

$$\mathbf{x}_t \in \mathbb{R}^{|V|}$$

$$\mathbf{e}_t, \mathbf{h}_t \in \mathbb{R}^d$$

$$\mathbf{W} \in \mathbb{R}^{d \times d}$$

$$\mathbf{U} \in \mathbb{R}^{d \times d}$$

$$\mathbf{E} \in \mathbb{R}^{d \times |V|}$$

~~$$\mathbf{V} \in \mathbb{R}^{|V| \times d}$$~~

We can use a single embedding matrix for both embedding and unembedding.

This sensibly reduces the number of trainable parameters required for the model.

RNNS FOR OTHER NLP TASKS



Sequence Labeling

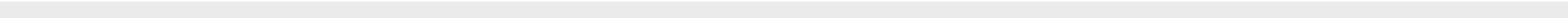
e.g., part-of-speech
tagging

Sequence Classification

e.g., sentiment analysis,
topic classification

Text generation

Encoder-decoder
architecture



SEQUENCE LABELING

Assign a label chosen from a small fixed set of labels to each element of an input sequence.

SEQUENCE LABELING

Assign a label chosen from a small fixed set of labels to each element of an input sequence.

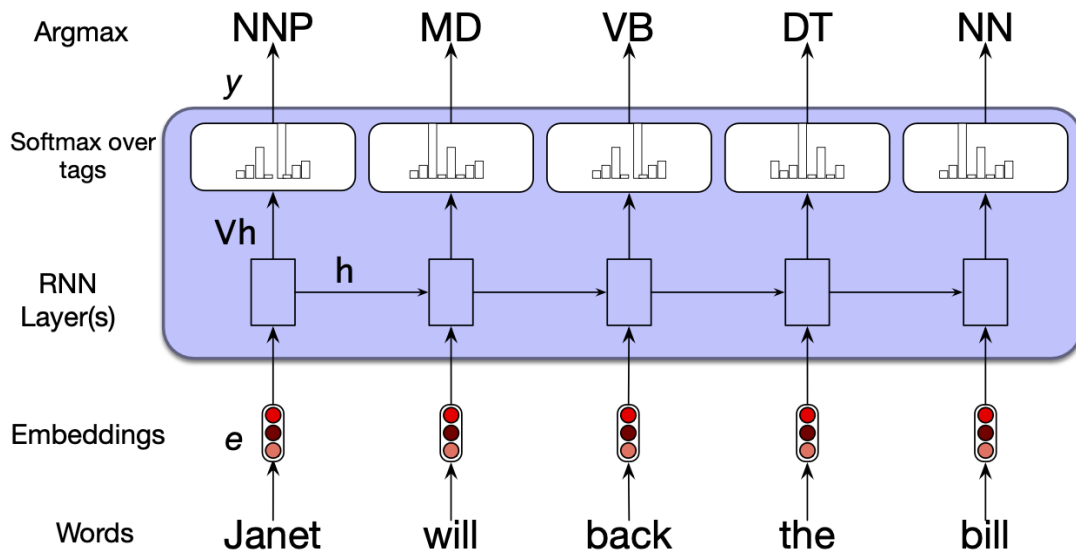
Example: **Part-of-speech (POS)** tagging
Assigning grammatical tags to each word in a sentence

SEQUENCE LABELING

Assign a label chosen from a small fixed set of labels to each element of an input sequence.

Example: **Part-of-speech (POS)** tagging

Assigning grammatical tags to each word in a sentence



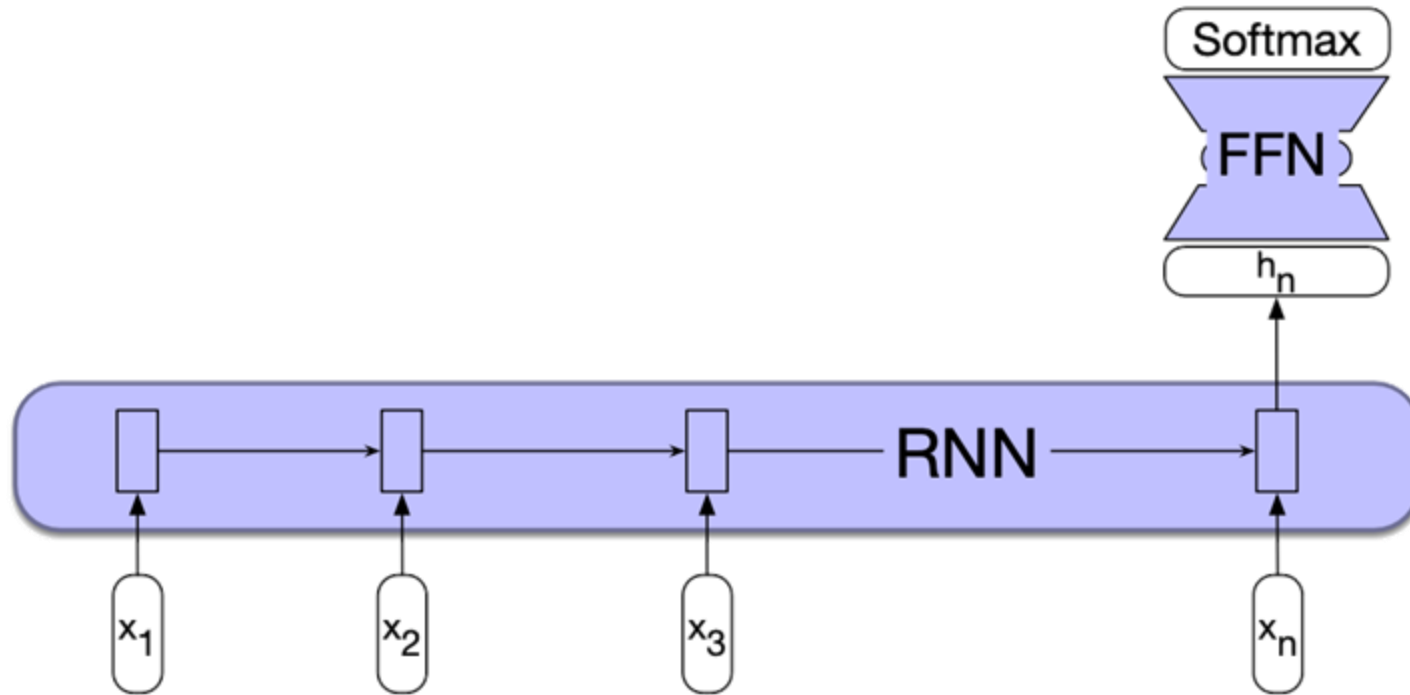
- Inputs to the RNN are word embeddings
- The output of the RNN at each step provides a distribution over the POS tagset
- Use cross-entropy loss during training

SEQUENCE CLASSIFICATION

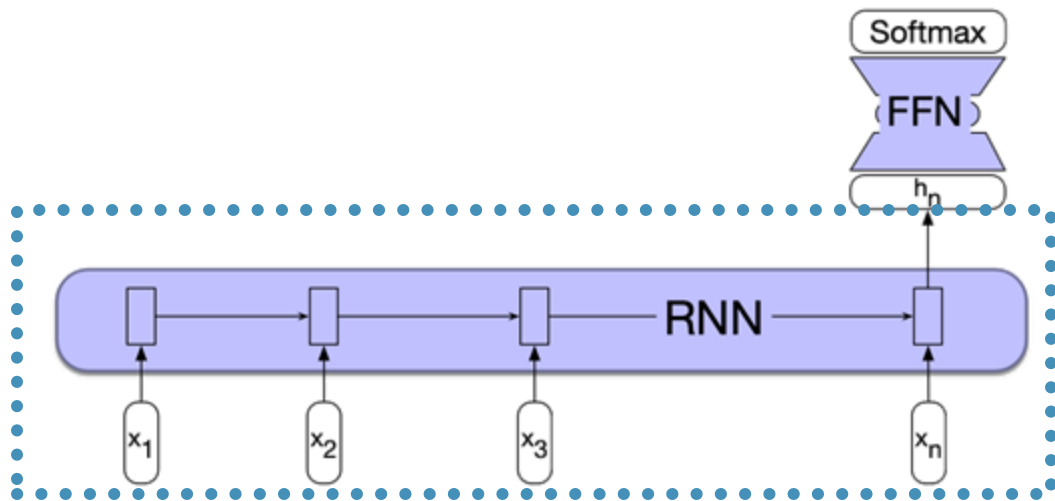
Classify an entire sequence, rather than the tokens within it (text classification).

Examples: spam detection, sentiment analysis.

SEQUENCE CLASSIFICATION



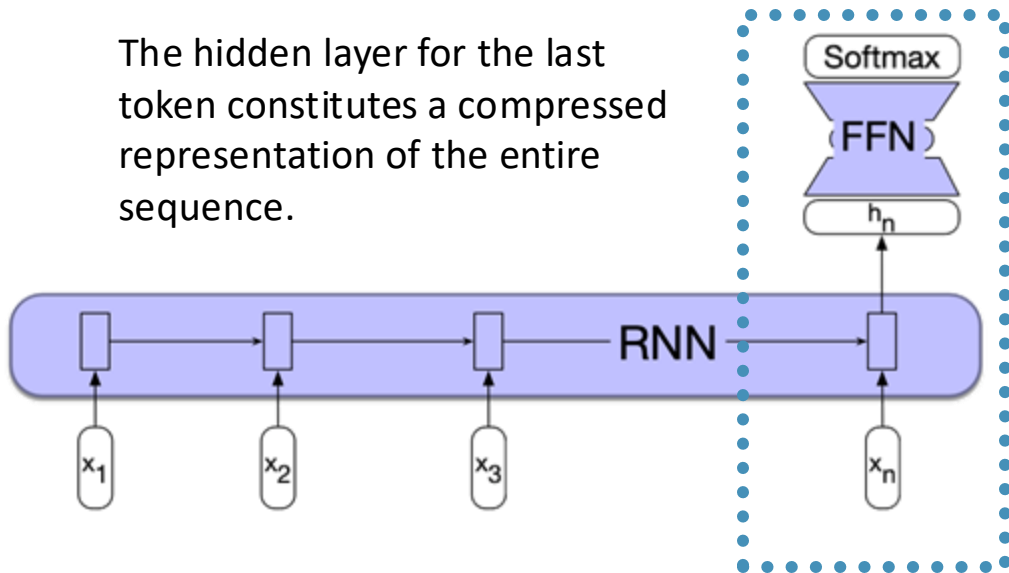
SEQUENCE CLASSIFICATION



We pass the text to be classified through the RNN one word at a time, generating a new hidden layer representation at each time step.

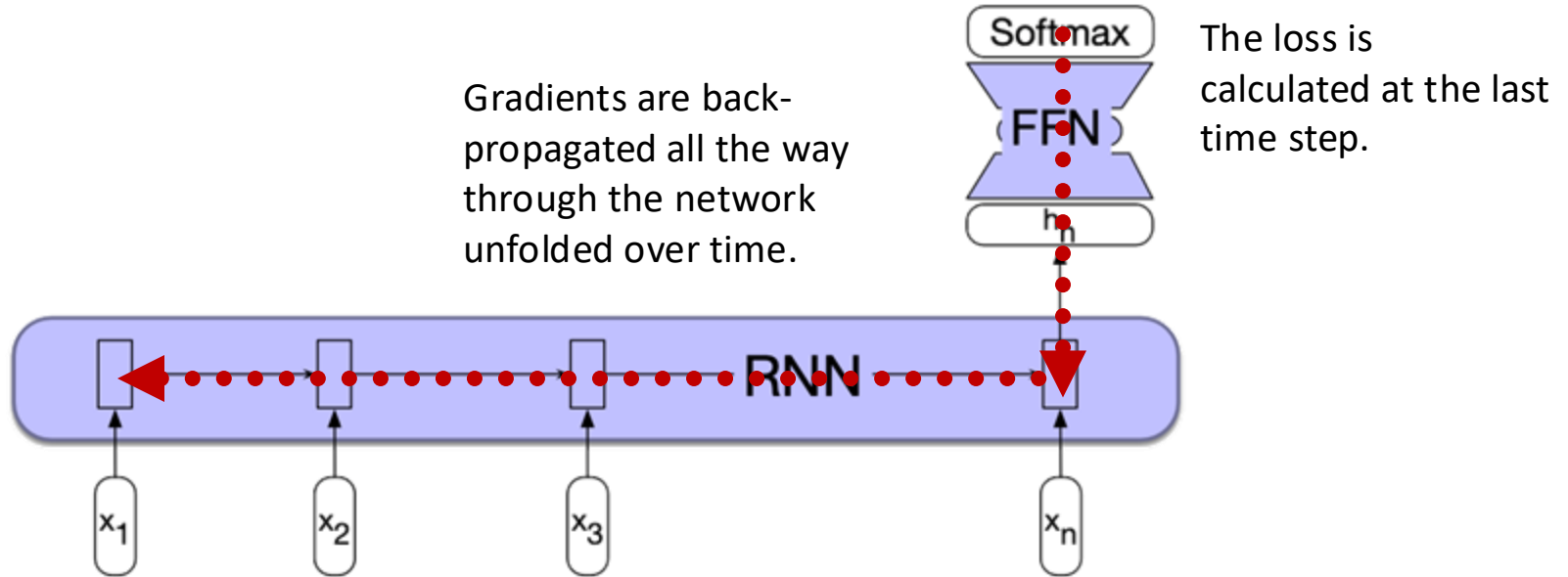
SEQUENCE CLASSIFICATION

The hidden layer for the last token constitutes a compressed representation of the entire sequence.



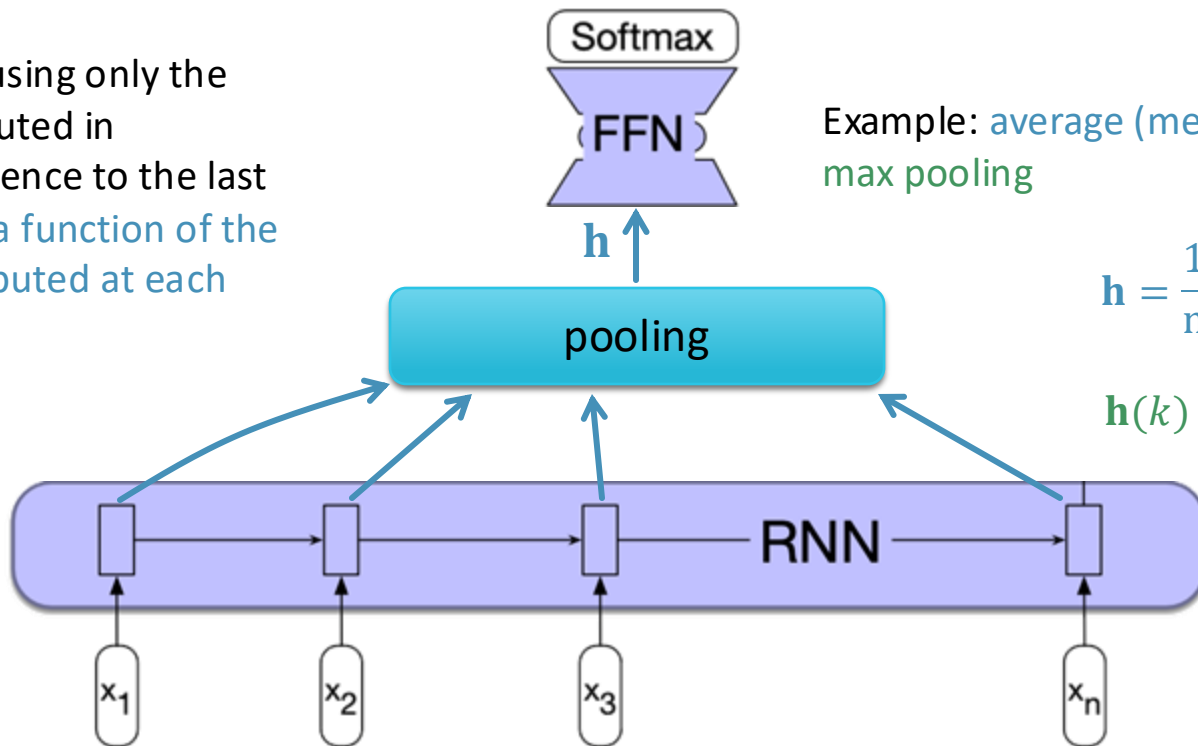
We pass the RNN state at the last time step to a feedforward network (the readout) that chooses a class via a softmax over the possible classes.

SEQUENCE CLASSIFICATION – TRAINING END-TO-END



SEQUENCE CLASSIFICATION – POOLING

Instead of using only the state computed in correspondence to the last token, use a function of the states computed at each time step.



Example: average (mean) pooling,
max pooling

$$\mathbf{h} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i$$

$$\mathbf{h}(k) = \max_{i=1}^n \mathbf{h}_i(k)$$

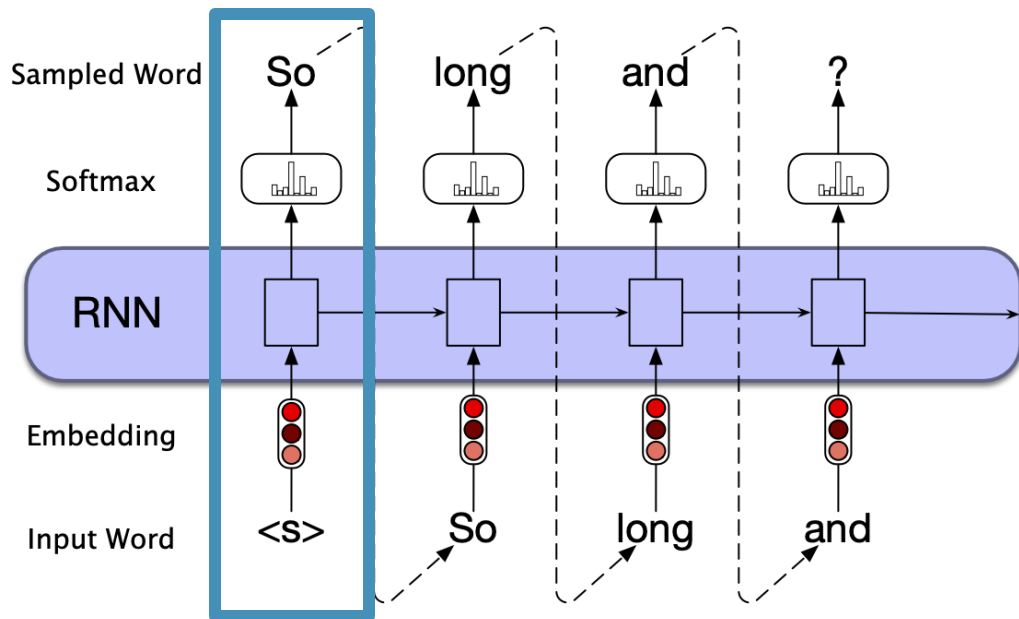
TEXT GENERATION

Autoregressive generation, or causal LM generation.

We use our RNN to generate text by sampling the next token, conditioned on the history of previous choices.

TEXT GENERATION

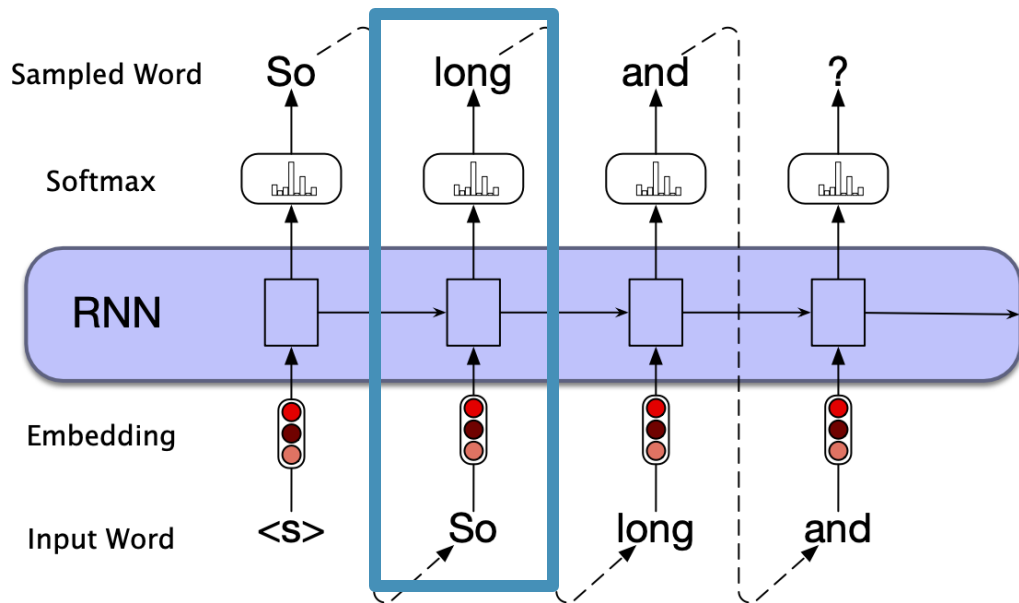
Autoregressive generation, or causal LM generation.



Sample a word in the output from the softmax distribution that results from using the **beginning of sentence marker $\langle s \rangle$** , as the first input.

TEXT GENERATION

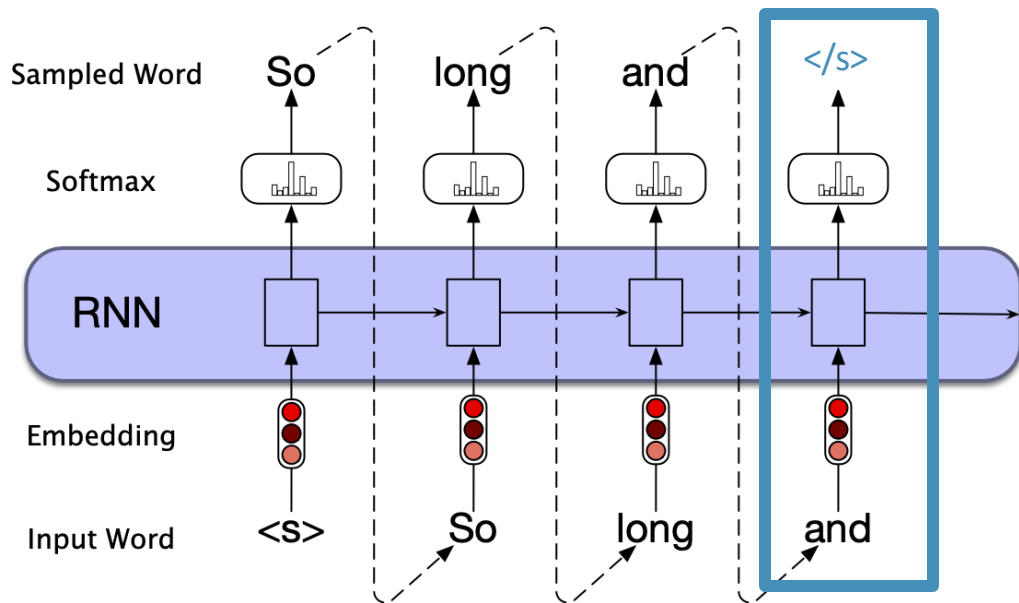
Autoregressive generation, or causal LM generation.



Use the word embedding for that first word as the input to the network at the next time step and proceed in the same way.

TEXT GENERATION

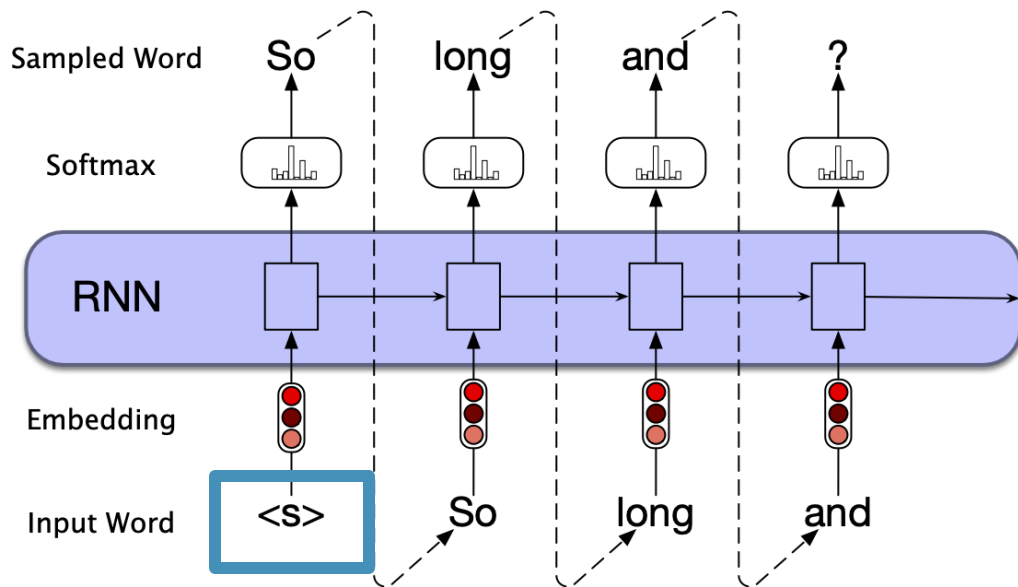
Autoregressive generation, or causal LM generation.



Continue until the end of sentence marker **</s>** is sampled (or a fixed length limit is reached).

TEXT GENERATION

Autoregressive generation, or causal LM generation.



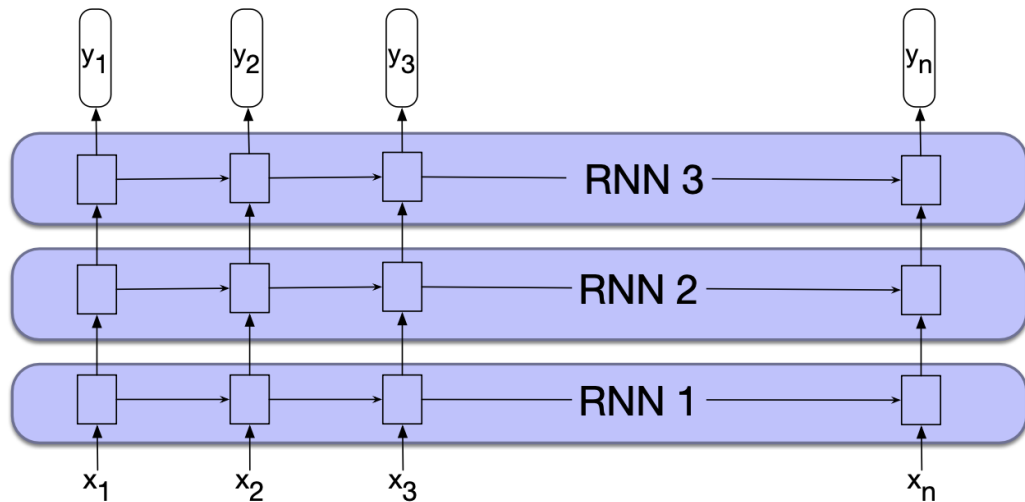
Instead of <s> to start, **we can use a richer task-appropriate context.**

Examples:

- Machine translation: use sentence in the source language
- Summarization: use the long text you want to summarize.

STACKED AND BIDIRECTIONAL RNN ARCHITECTURES

STACKED RNNs (DEEP RNNs)



The output from one RNN is used as (driving) input to the following RNN in the stack.

Stacking RNN layers induces representations at different levels of (temporal) abstraction across layers.

The optimal number of layers is task-dependent.

HOW TO CONSTRUCT DEEP RNNs

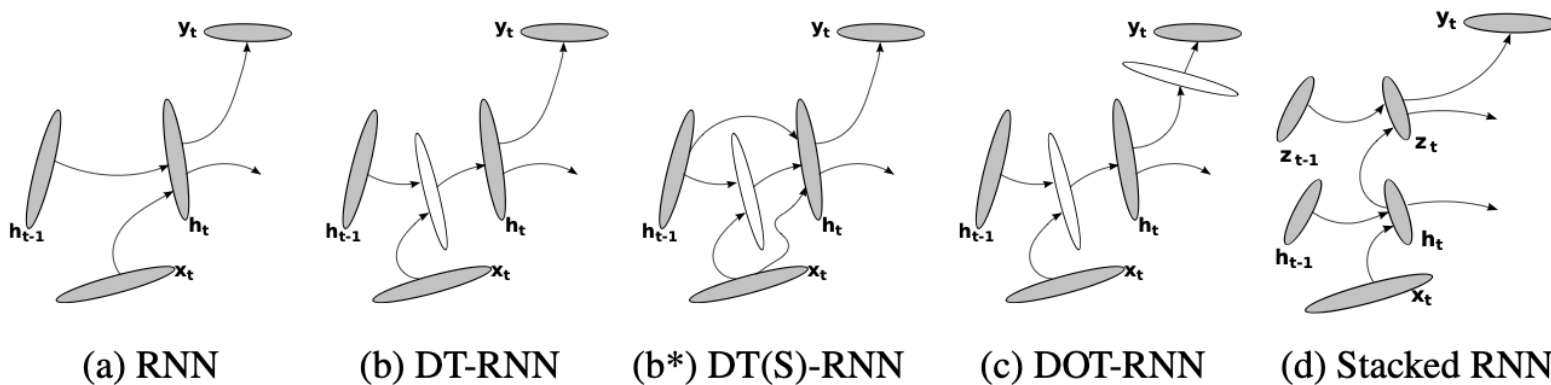
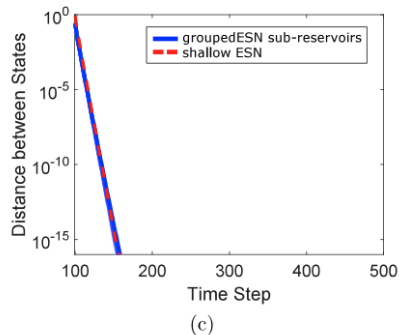
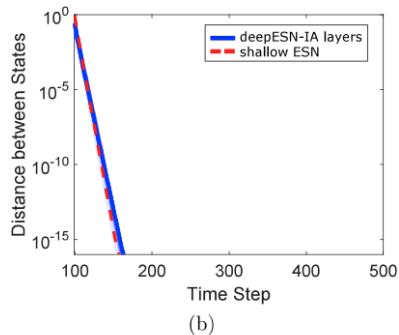
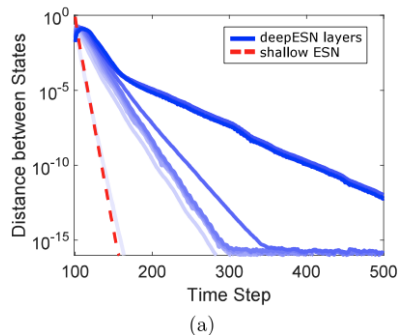


Figure 2: Illustrations of four different recurrent neural networks (RNN). (a) A conventional RNN. (b) Deep Transition (DT) RNN. (b*) DT-RNN with shortcut connections (c) Deep Transition, Deep Output (DOT) RNN. (d) Stacked RNN



THE INTRINSIC ROLE OF DEPTH IN RNNs



Gallicchio, Claudio, Alessio Micheli, and Luca Pedrelli. "Deep reservoir computing: A critical experimental analysis." *Neurocomputing* 268 (2017): 87-99.

Figure 5: Distance between perturbed and unperturbed states on the Artificial dataset for the considered RC architectures, with $a = 0.55$ and $\rho = 0.9$ for every layer (or sub-reservoir) and using IP learning. Continuous blue lines correspond to layers in deep RC networks (darker colors for higher layers) and to sub-reservoirs in groupedESN. Dashed red lines correspond to the shallow ESN with the same number of total reservoir units and hyper-parametrization. **a:** deepESN, **b:** deepESN-IA, **c:** groupedESN.

BIDIRECTIONAL RNNS

Vanilla RNNs use information from the left (previous/past) context to make predictions at the current time t .

In many cases, when dealing with text, we have access to the entire input sequence.

Idea: run two separate RNNs

- one forwards (left-to-right)
- one backwards (right-to-left)

concatenate the representations

BIDIRECTIONAL RNNS

Forward RNN (left-to-right)

The hidden state at a given time t is a function of the inputs $x_1, x_2, \dots, x_t$.

It represents the context of the network to the left of the current time.

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t)$$

BIDIRECTIONAL RNNS

Backward RNN (right-to-left)

The hidden state at a given time t is a function of the inputs $x_n, x_{n-1}, \dots, x_t$ (*i.e., the RNN is trained on the reversed input sequence*).

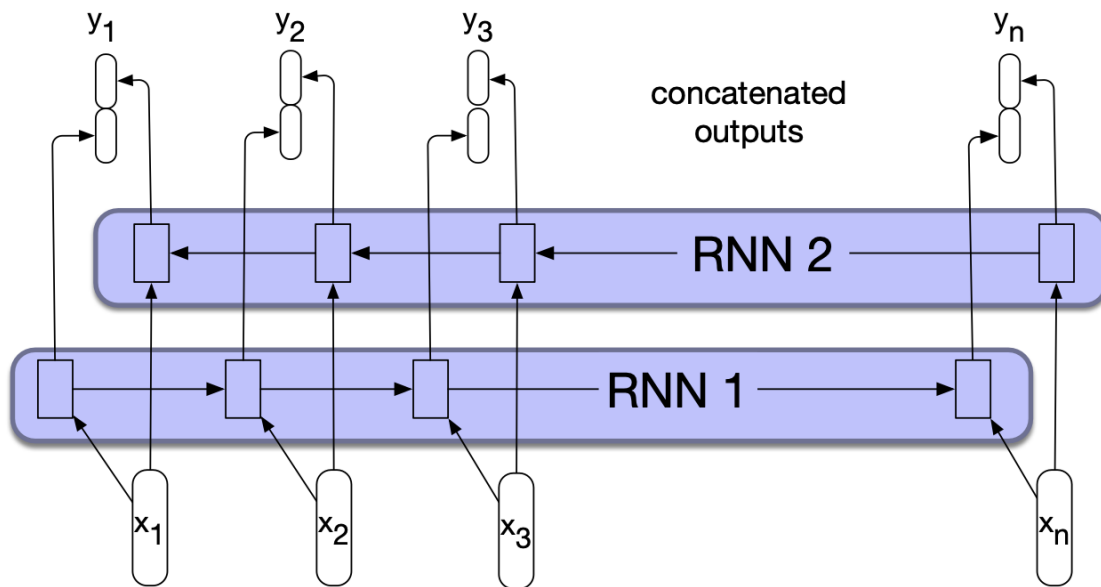
It represents the context of the network to the right of the current time.

$$\mathbf{h}_t^f = \text{RNN}_{\text{backward}}(\mathbf{x}_n, \mathbf{x}_{n-1}, \dots, \mathbf{x}_t)$$

BIDIRECTIONAL RNNs

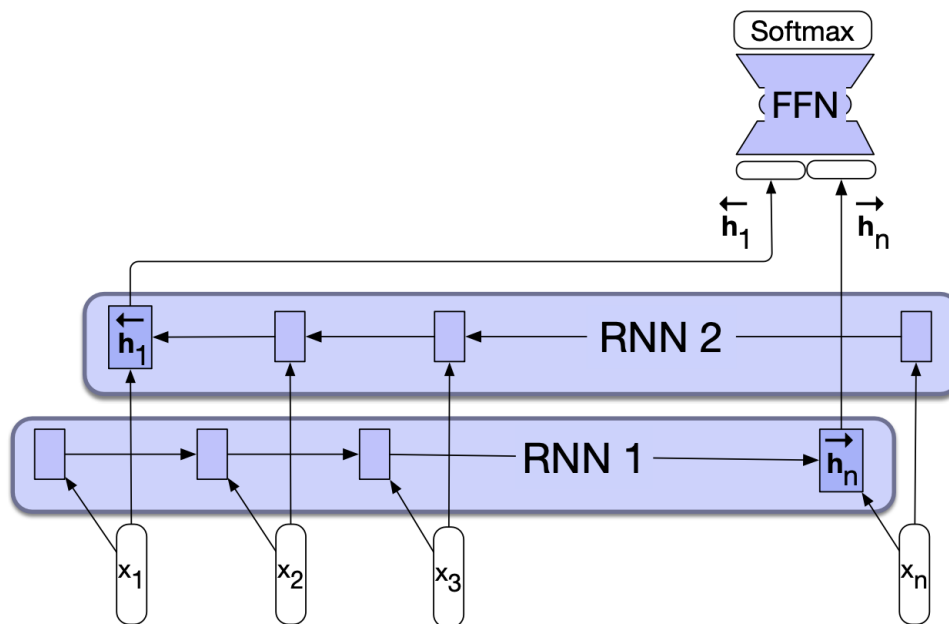
Bidirectional RNN: combine the two

$$\mathbf{h}_t = [\mathbf{h}_t^f; \mathbf{h}_t^b]$$



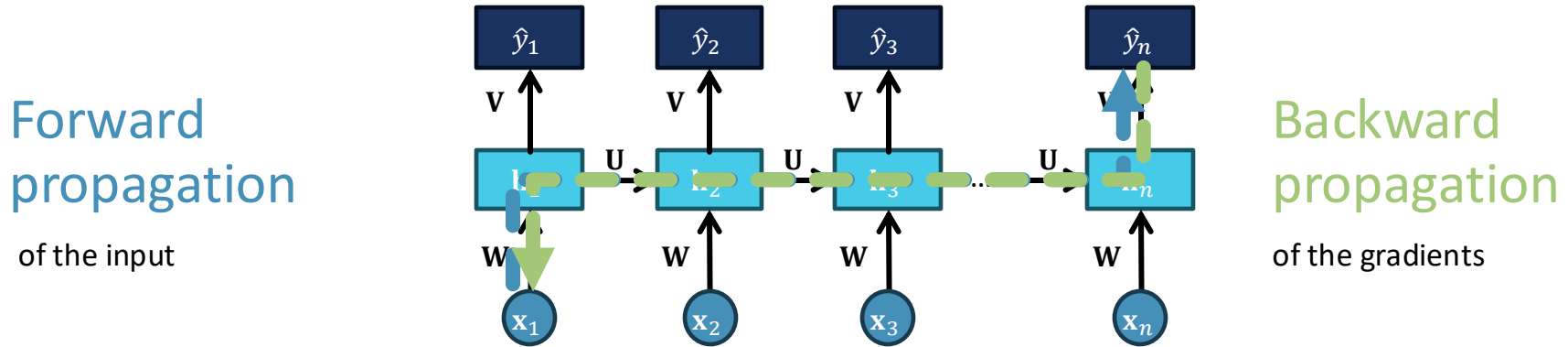
BIDIRECTIONAL RNNs

Bidirectional RNN for sequence classification



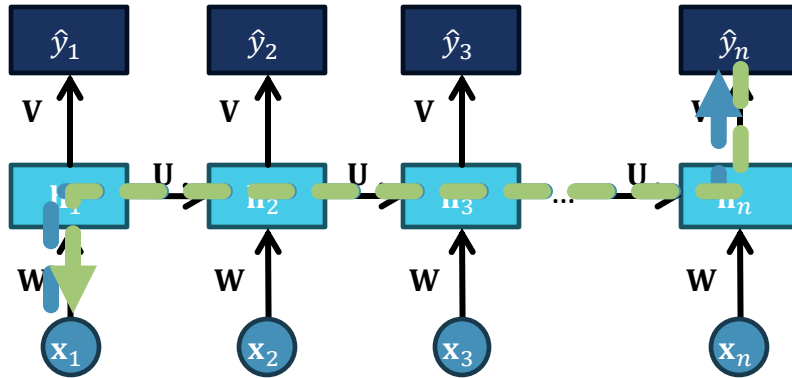
LSTM AND GATED RNN ARCHITECTURES

A recurrent layer adds another type of “deep” dimension.



shows the same issues as deep feed forward neural nets

shows the same issues as deep feed forward neural nets



potential instabilities in both

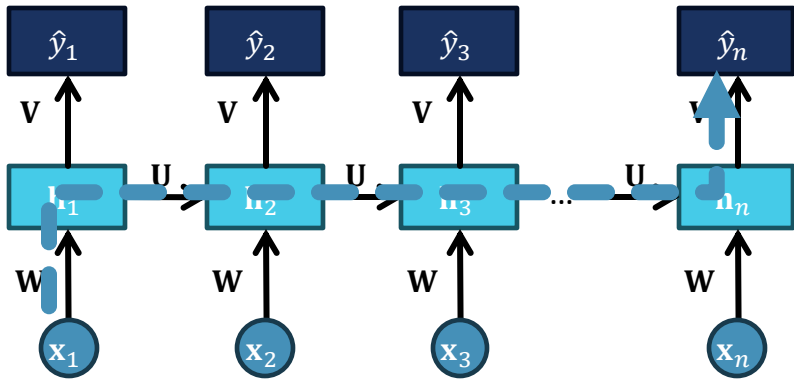
Forward
propagation

of the input

Backward
propagation

of the gradients

shows the same issues as deep feed forward neural nets



potential instabilities in both

Forward
propagation

of the input

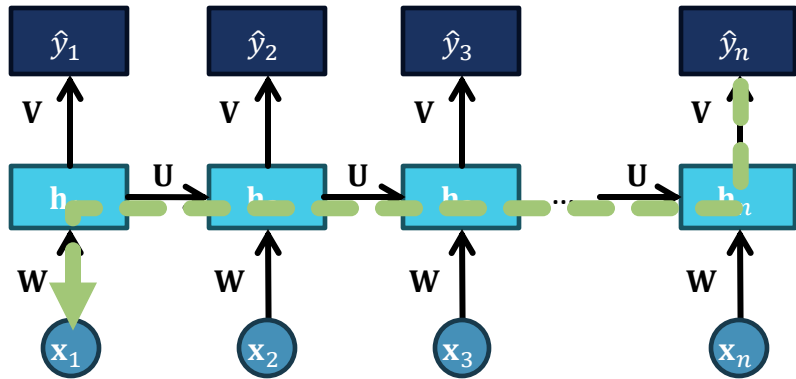
Backward
propagation

of the gradients

The influence of inputs far in the past vanishes in the current state.

The influence of input perturbations can explode asymptotically in the state.

shows the same issues as deep feed forward neural nets



potential instabilities in both

Forward propagation of the input
Backward propagation of the gradients

Gradient might vanish or explode

It's difficult to train RNNs on long-term dependencies



Training RNNs is
difficult.

On the difficulty of training recurrent neural networks

Razvan Pascanu

Université de Montréal, 2920, chemin de la Tour, Montréal, Québec, Canada, H3T 1J8

PASCANUR@IRO.UMONTREAL.CA

Tomas Mikolov

Speech@FIT, Brno University of Technology, Brno, Czech Republic

T.MIKOLOV@GMAIL.COM

Yoshua Bengio

Université de Montréal, 2920, chemin de la Tour, Montréal, Québec, Canada, H3T 1J8

YOSHUA.BENGIO@UMONTREAL.CA

Abstract

There are two widely known issues with properly training recurrent neural networks, the *vanishing* and the *exploding* gradient problems detailed in Bengio *et al.* (1994). In this paper we attempt to improve the understanding of the underlying issues by exploring these problems from an analytical, a geometric and a dynamical systems perspective. Our analysis is used to justify a simple yet effective solution. We propose a gradient norm clipping strategy to deal with exploding gradients and a soft constraint for the vanishing gradients problem. We validate empirically our hypothesis and proposed solutions in the experimental section.

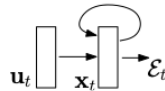


Figure 1. Schematic of a recurrent neural network. The recurrent connections in the hidden layer allow information to persist from one input to another.

1.1. Training recurrent networks

A generic recurrent neural network, with input $\mathbf{u}_t$ and state $\mathbf{x}_t$ for time step t , is given by:

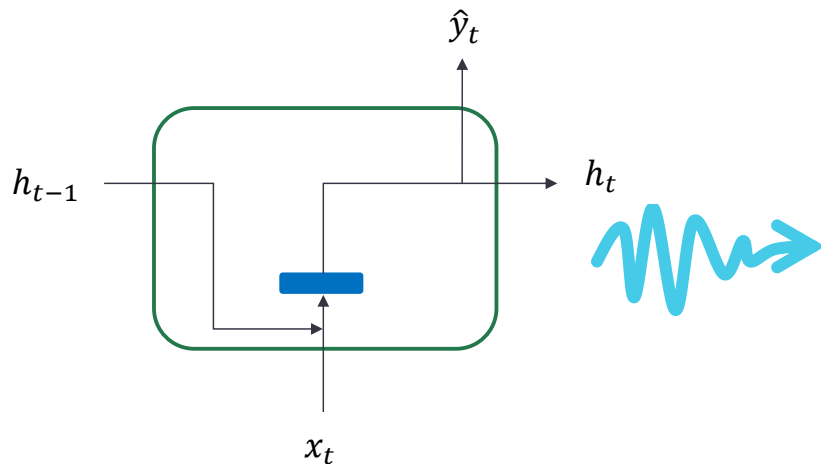
$$\mathbf{x}_t = F(\mathbf{x}_{t-1}, \mathbf{u}_t, \theta) \quad (1)$$

In the theoretical section of this paper we will sometimes make use of the specific parametrization: ¹

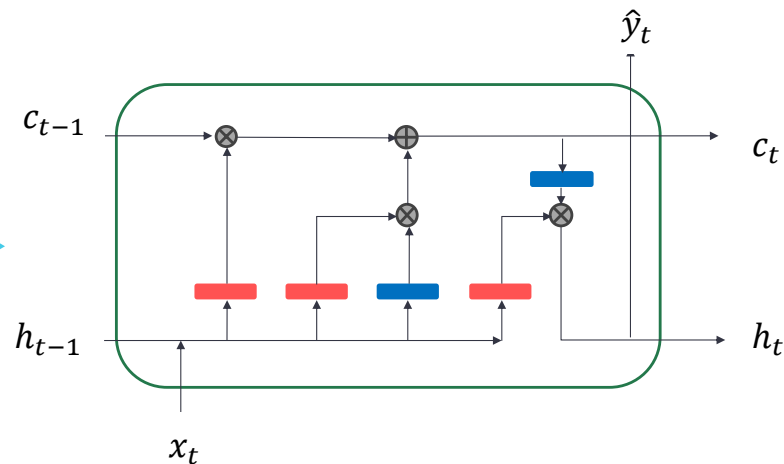
$$\mathbf{x}_t = \mathbf{W}_{rec}\sigma(\mathbf{x}_{t-1}) + \mathbf{W}_{in}\mathbf{u}_t + \mathbf{b} \quad (2)$$

Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio. "On the difficulty of training recurrent neural networks." International conference on machine learning. Pmlr, 2013.

NEED FOR ADVANCED ARCHITECTURES



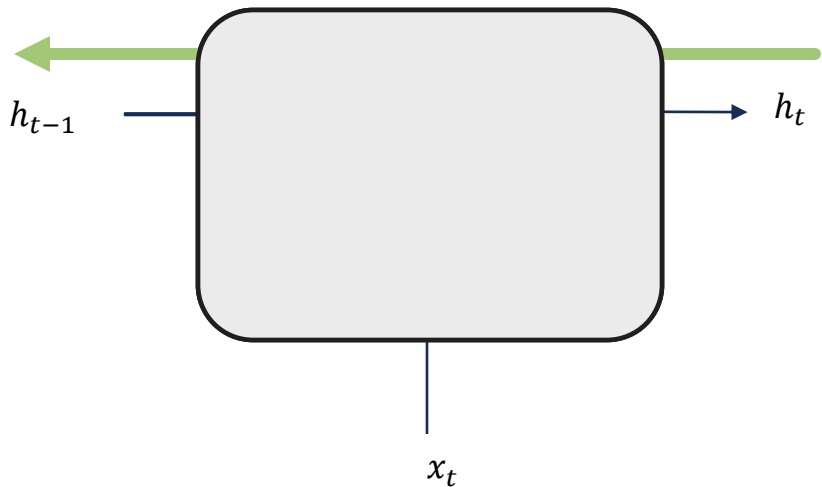
Standard RNN cell



Long short-term memory (LSTM) cell

Overcoming vanishing & exploding gradient problems in RNNs & effectively learn long-range dependencies.

LEAKY RNN



create an effective path through time for information flowing

$$\mathbf{s}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1})$$

$$\mathbf{h}_t = \alpha \mathbf{h}_{t-1} + (1 - \alpha) \mathbf{s}_t$$

leaky constant

LONG SHORT-TERM MEMORY (LSTM)

The network can learn what to store and what to throw away from the state

2 types of “state” information

h_t – short-term state

- Contains information on the current time-step
- Used for making predictions / further processing

c_t – long-term state context vector

- Internal memory of the LSTM unit
- Carries information across time-steps
- Allows information retention over long sequences

GATES

i_t input gate

determines which **new information** should be **stored** in the cell state

f_t forget gate

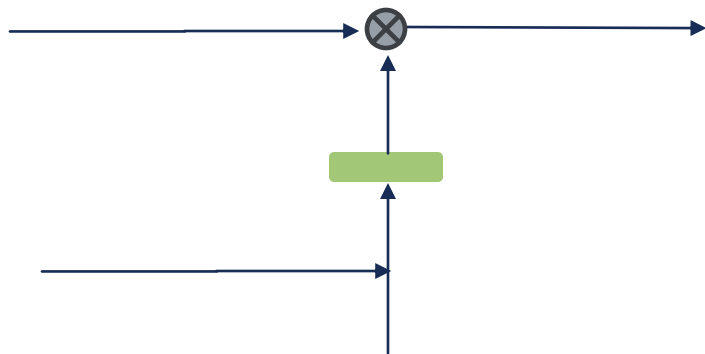
determines what information from the cell state should be **discarded** or forgotten

o_t output gate

controls which part of the cell state should be exposed to the **output**

GATE

standard NN layer with sigmoid activation + pointwise multiplication



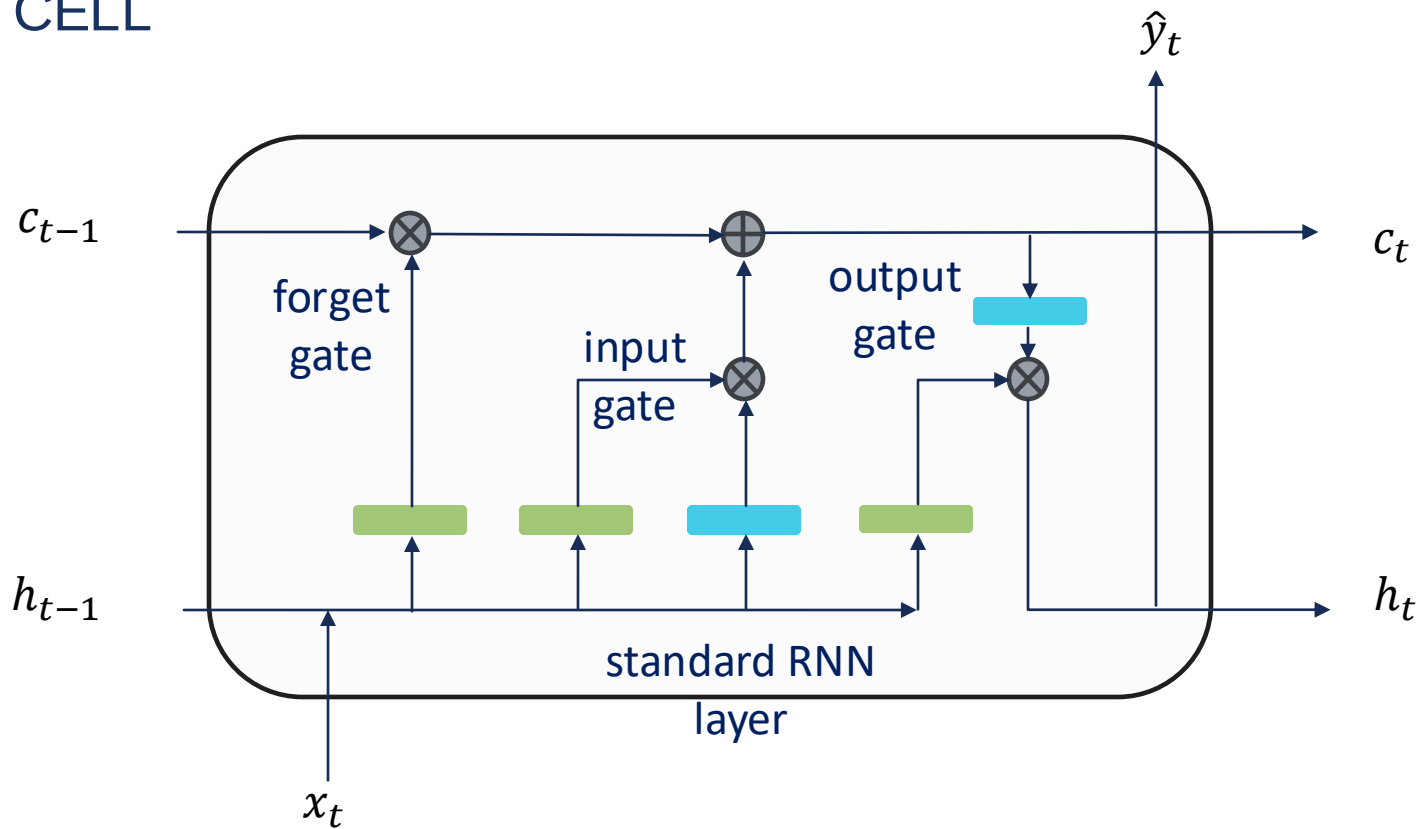
anything that passes through the gate is forced to be between 0 and 1

modulate how much of the input to the gate should pass through

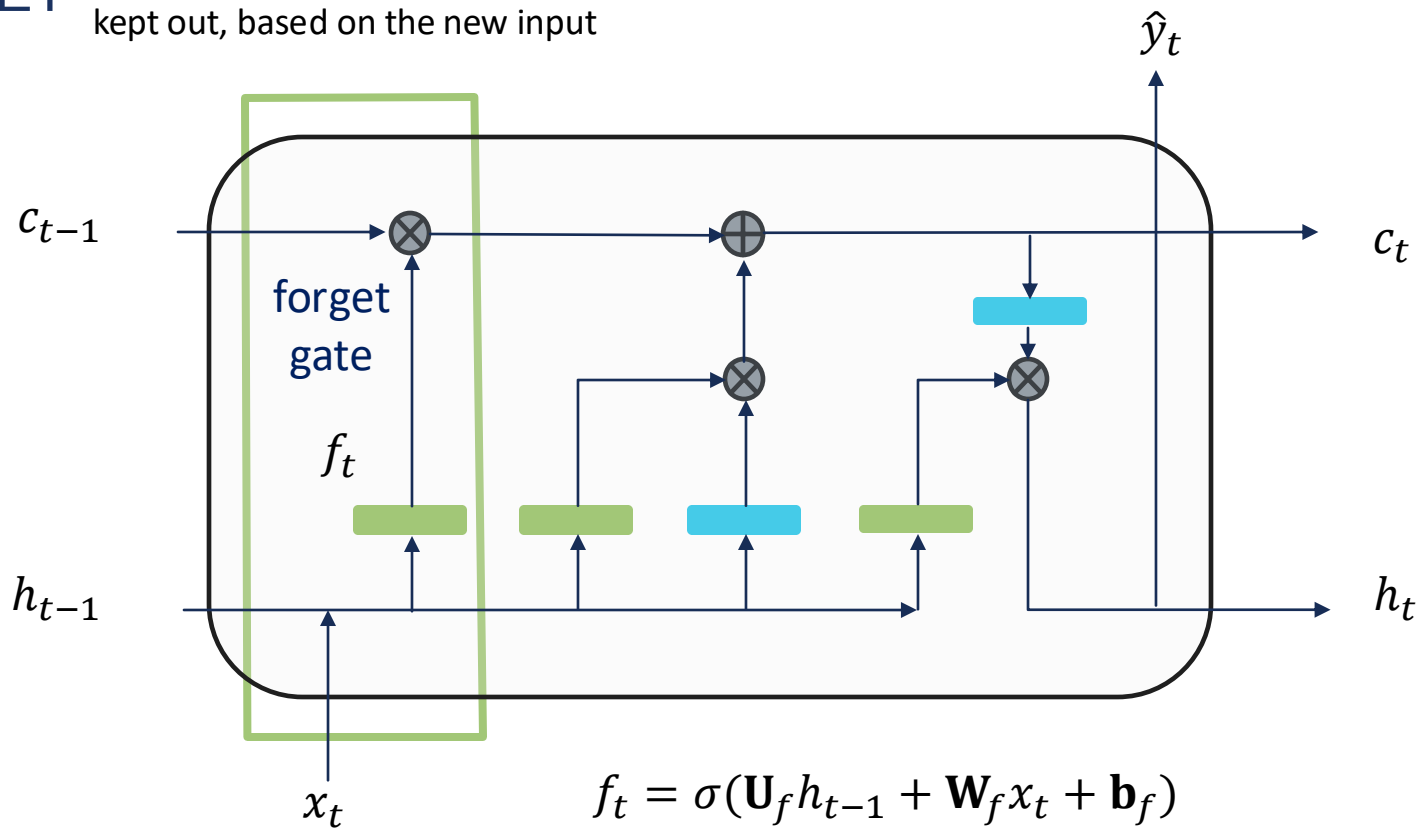
- everything 1
- nothing 0



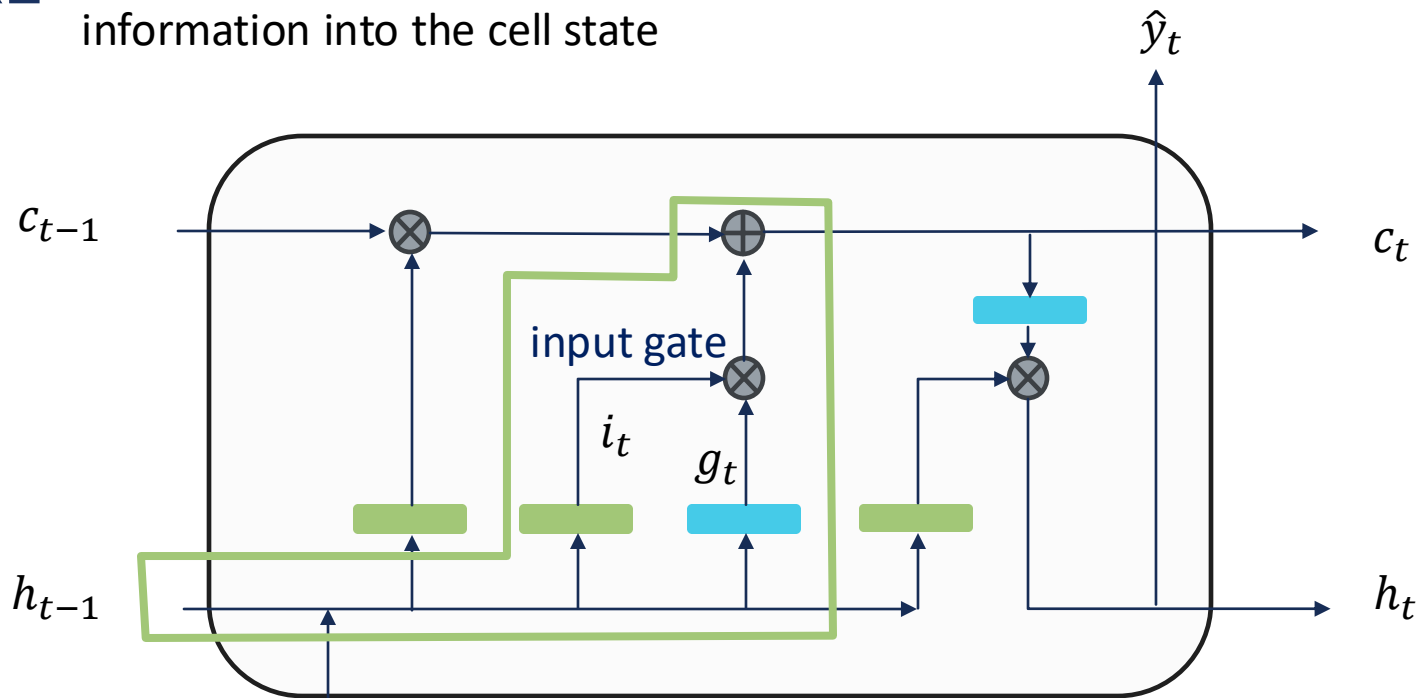
LSTM CELL



FORGET how much of the old state should pass in or kept out, based on the new input



STORE Modulates the inclusion of new input information into the cell state



Which part of the transformed input (through g_t) should go into the cell state

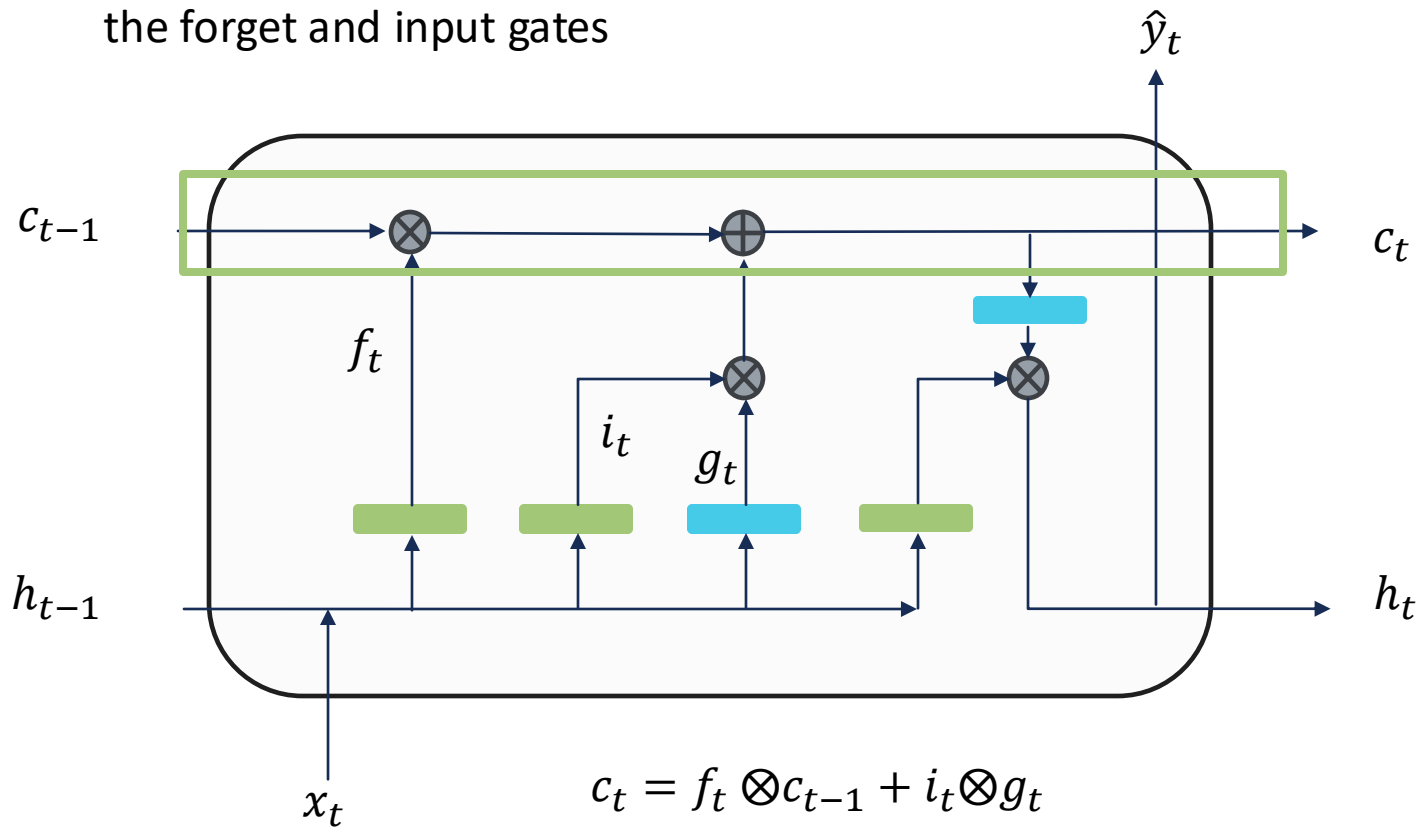
$$g_t = \tanh(\mathbf{U}_g h_{t-1} + \mathbf{W}_g x_t + \mathbf{b}_g)$$

Same as in vanilla RNN

$$i_t = \sigma(\mathbf{U}_i h_{t-1} + \mathbf{W}_i x_t + \mathbf{b}_i)$$

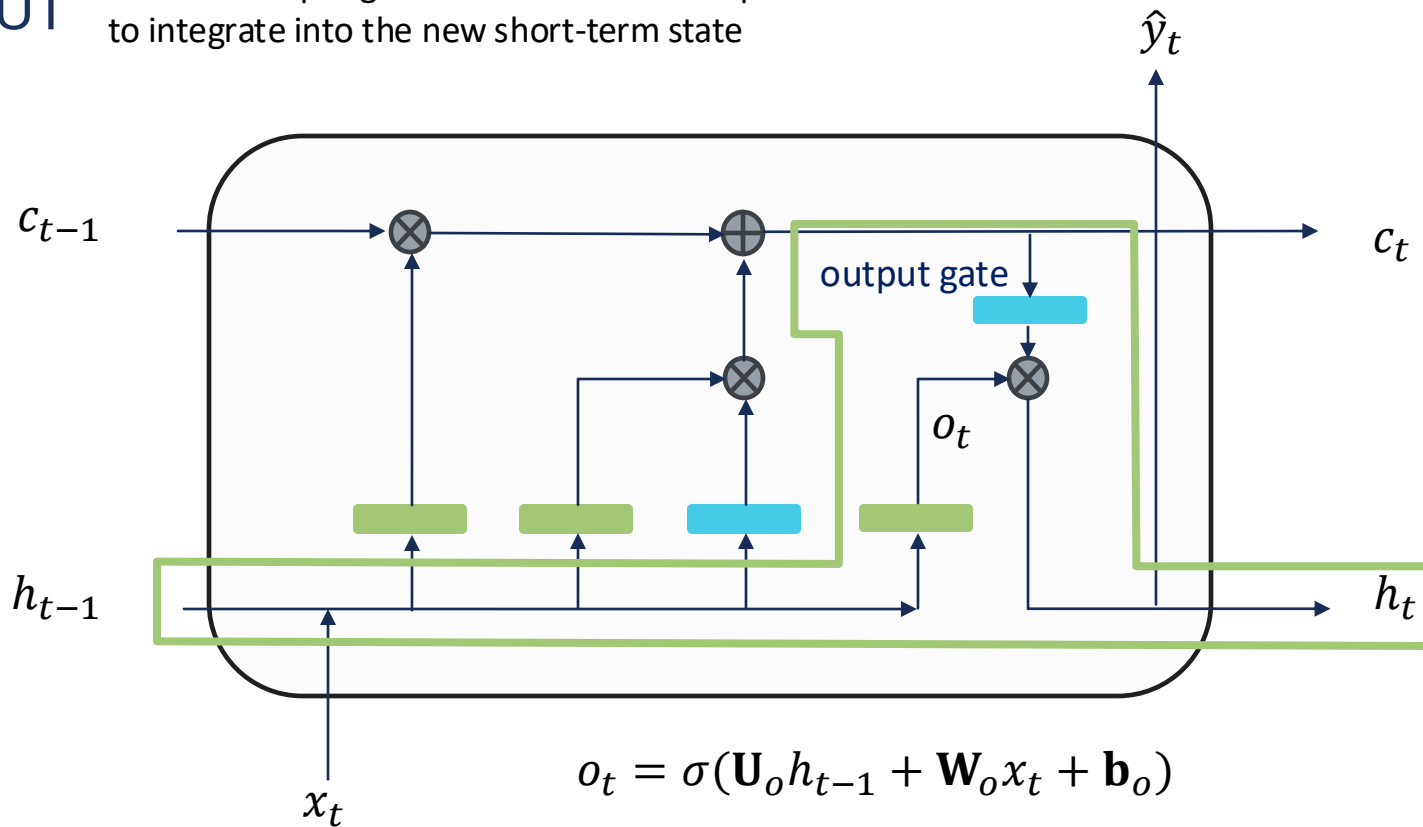
UPDATE

The cell state is updated based on the operations of the forget and input gates



OUTPUT

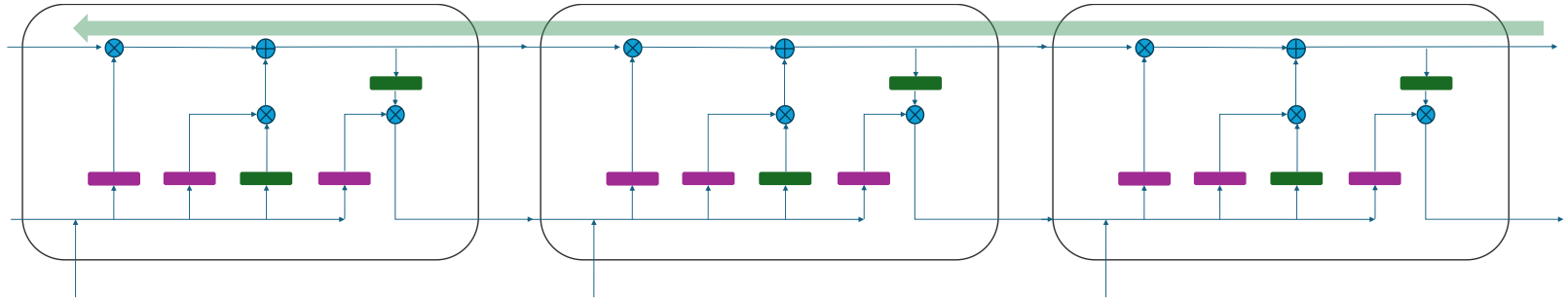
Uses the output gate to determine which part of the cell state to integrate into the new short-term state



$$o_t = \sigma(\mathbf{U}_o h_{t-1} + \mathbf{W}_o x_t + \mathbf{b}_o)$$

$$h_t = o_t \otimes \tanh(c_t)$$

all these gating mechanisms work to allow for a kind of **gradient computation flow without interruptions**



$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t + \mathbf{b}_f)$$

$$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t + \mathbf{b}_g) \quad \text{vanilla RNN}$$

$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t + \mathbf{b}_i)$$

$$\mathbf{c}_t = \mathbf{f}_t \otimes \mathbf{c}_{t-1} + \mathbf{i}_t \otimes \mathbf{g}_t \quad \text{long-term state (context)}$$

$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \otimes \tanh(\mathbf{c}_t) \quad \text{short-term state}$$

LSTM WRAP UP

- Maintain a separate “long-term” memory cell state $c(t)$
- Use gates to control the flow of information
 - forget gate: get rid of irrelevant information
 - input gate: store relevant information from the current input
 - output gate: return a filtered version of the cell state
- Pro: BPTT with uninterrupted gradient flow
- Con: training is more computationally intensive

GATED RECURRENT UNIT (GRU) Simpler alternative to LSTM cell

There is only one state: $\mathbf{h}(t)$

There are only two gates

r_t reset gate

how much of the previous
state participates in the
update with the new input

z_t update gate

how much of the previous state
contributes to the new state

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)

GATED RECURRENT UNIT (GRU) Simpler alternative to LSTM cell

$$\mathbf{z}_t = \sigma(\mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{W}_z \mathbf{x}_t + \mathbf{b}_z)$$

$$\mathbf{r}_t = \sigma(\mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{W}_r \mathbf{x}_t + \mathbf{b}_r)$$

$$\mathbf{g}_t = \tanh(\mathbf{U}_a (\mathbf{r}_t \otimes \mathbf{h}_{t-1}) + \mathbf{W}_a \mathbf{x}(t) + \mathbf{b}_a)$$

$$\mathbf{h}_t = \mathbf{z}_t \otimes \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \otimes \mathbf{g}_t$$

how much to take from
the previous state in the
new state update

vanilla
RNN

how much to update the state

ENCODER-DECODER MODEL WITH RNNS

ENCODER-DECODER MODEL

Take an input sequence and map it (translate it) to an output sequence that:

- can be of a different length than the input
- does not align with the input sequence in a step-by-step way

ENCODER-DECODER MODEL

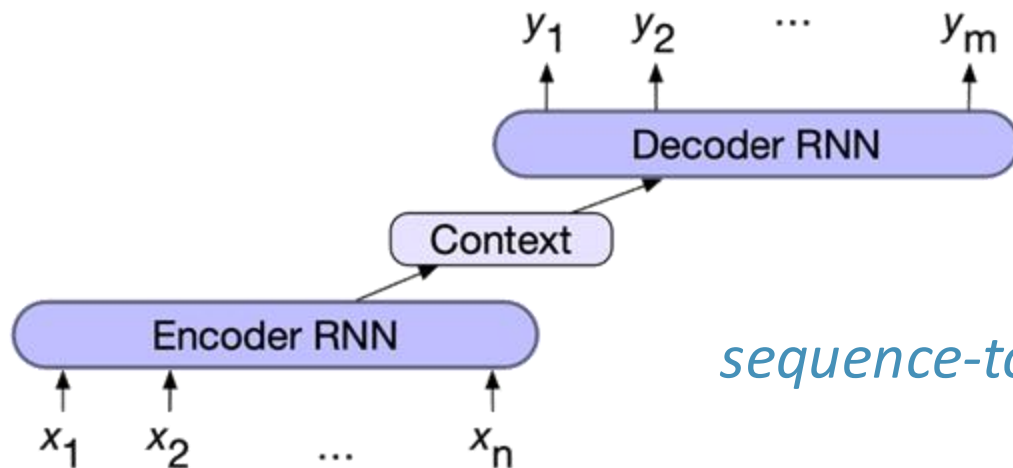
Take an input sequence and map it (translate it) to an output sequence that:

- can be of a different length than the input
- does not align with the input sequence in a step-by-step way

A suitable approach for *machine translation*, where

- the input and the output texts can have different lengths
- the mapping between a token in the input and a token in the output can be indirect.

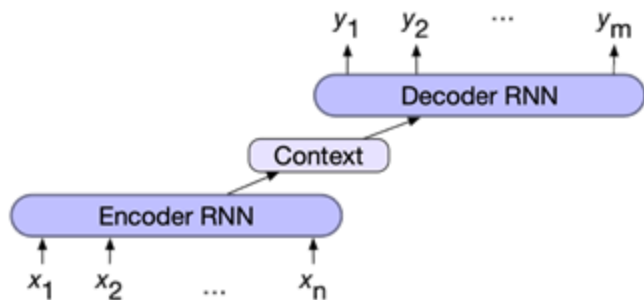
ENCODER-DECODER



sequence-to-sequence networks

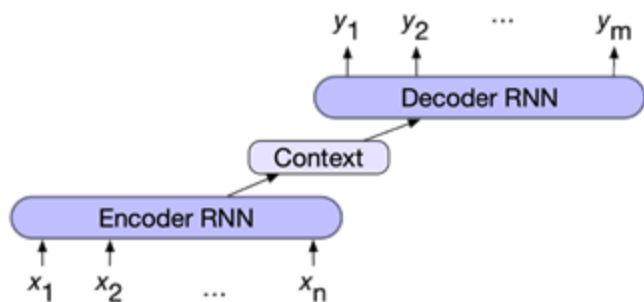
Summarization, question answering,
dialogue, machine translation...

ENCODER-DECODER KEY IDEA



The **encoder** network takes in input a sequence and creates a contextualized representation, i.e., the context.

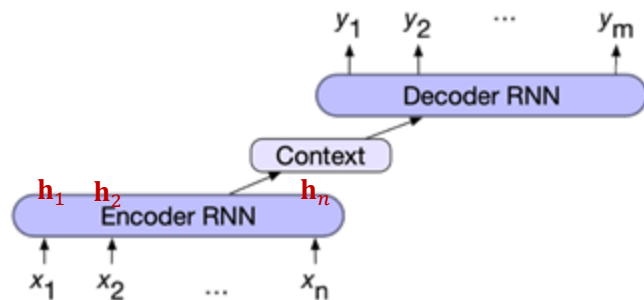
ENCODER-DECODER KEY IDEA



The **decoder** is fed by the context and generates a task specific output sequence.

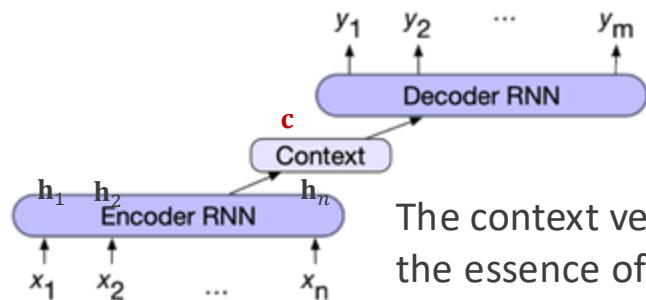
The encoder network takes in input a sequence and creates a contextualized representation, i.e., the context.

ENCODER-DECODER KEY IDEA



The encoder accepts in input sequence $\mathbf{x}_{1:n}$ and generates a corresponding sequence of contextualized representations $\mathbf{h}_{1:n}$ (e.g., by employing an LSTM)

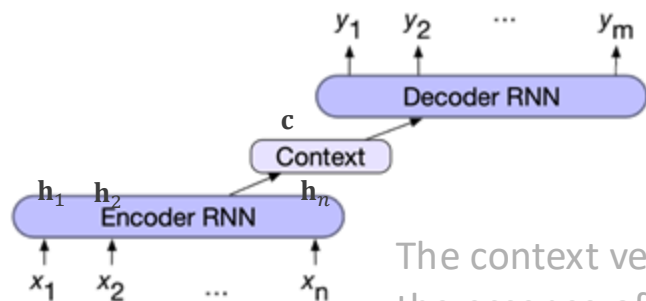
ENCODER-DECODER KEY IDEA



The context vector $\mathbf{c}$ is a function of $\mathbf{h}_{1:n}$ and conveys the essence of the input to the decoder

The encoder accepts in input sequence $\mathbf{x}_{1:n}$ and generates a corresponding sequence of contextualized representations $\mathbf{h}_{1:n}$ (e.g., by employing an LSTM)

ENCODER-DECODER KEY IDEA



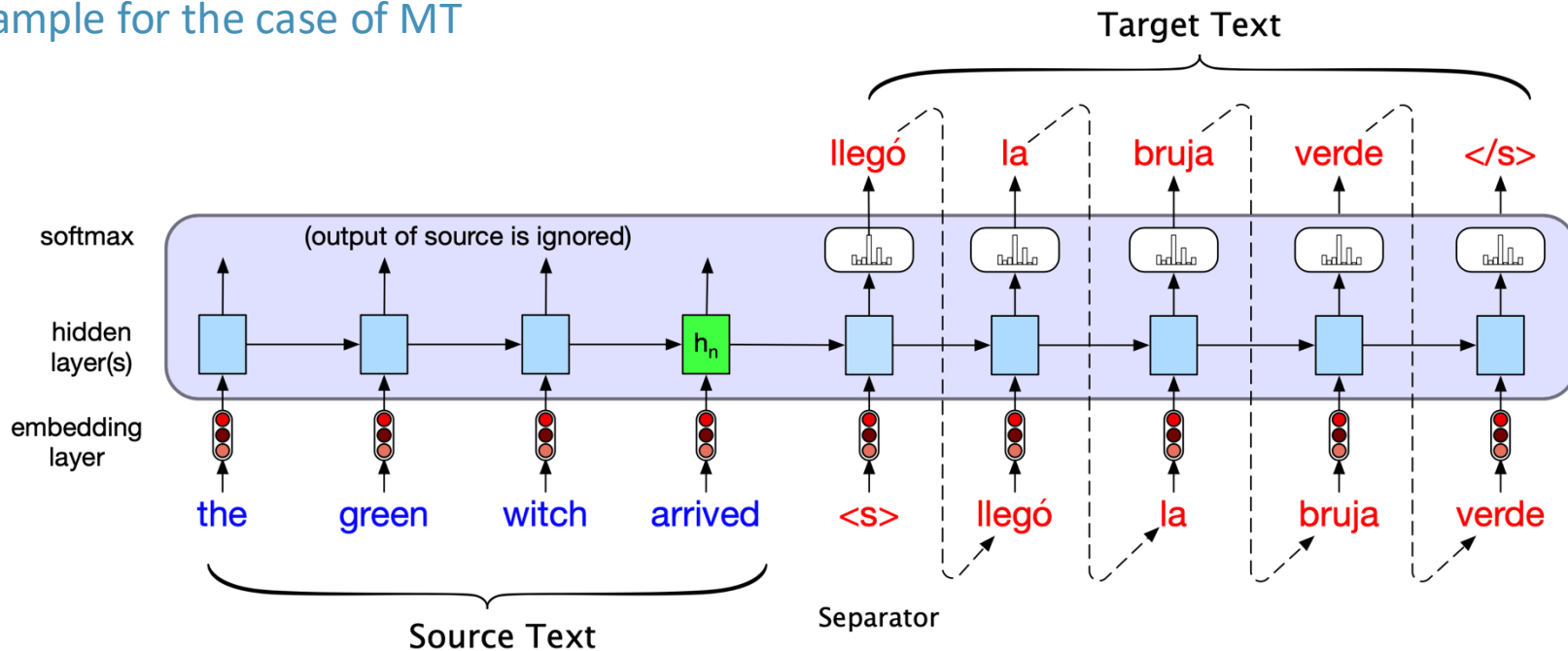
The decoder accepts c in input and generates an arbitrary length sequence of hidden states $\mathbf{h}'_{1:m}$ from which a sequence of outputs $\mathbf{y}_{1:m}$ can be obtained.

The context vector c is a function of $\mathbf{h}_{1:n}$ and conveys the essence of the input to the decoder

The encoder accepts in input sequence $\mathbf{x}_{1:n}$ and generates a corresponding sequence of contextualized representations $\mathbf{h}_{1:n}$ (e.g., by employing an LSTM)

ENCODER-DECODER KEY IDEA

Example for the case of MT



ENCODER

Operates like a conventional RNN-based neural architecture, to progressively encode the entire input sequence in its states.

The encoding process is stopped once we reach a sentence separation token <s>.

ENCODER

Operates like a conventional RNN-based neural architecture, to progressively encode the entire input sequence in its states.

The encoding process is stopped once we reach a sentence separation token <s>.

$$\mathbf{h}_t^e = g(\mathbf{h}_{t-1}^e, \mathbf{x}_t)$$

g implements the recurrent component of an RNN

often, the encoder is a stacked bi-directional LSTM network

At the end of the input sequence, $\mathbf{h}_n^e$ is used as the context, and serves as the initial state for the decoder RNN, $\mathbf{h}_0^d$

DECODER

Uses the context to initialize the first hidden state.

Generates a sequence of outputs in an autoregressive way.

- Each hidden state is conditioned on the previous hidden state and the output generated in the previous time.
- Or, the hidden state can be conditioned also on the context at each time step.

The generation is stopped once an end-of-sentence token `</s>` is generated.

DECODER

Uses the context to initialize the first hidden state.

Generates a sequence of outputs in an autoregressive way.

- Each hidden state is conditioned on the previous hidden state and the output generated in the previous time.
- Or, the hidden state can be conditioned also on the context at each time step.

The generation is stopped once an end-of-sentence token </s> is generated.

$$\mathbf{c} = \mathbf{h}_n^e$$

$$\mathbf{h}_0^d = \mathbf{c}$$

hidden RNN layer

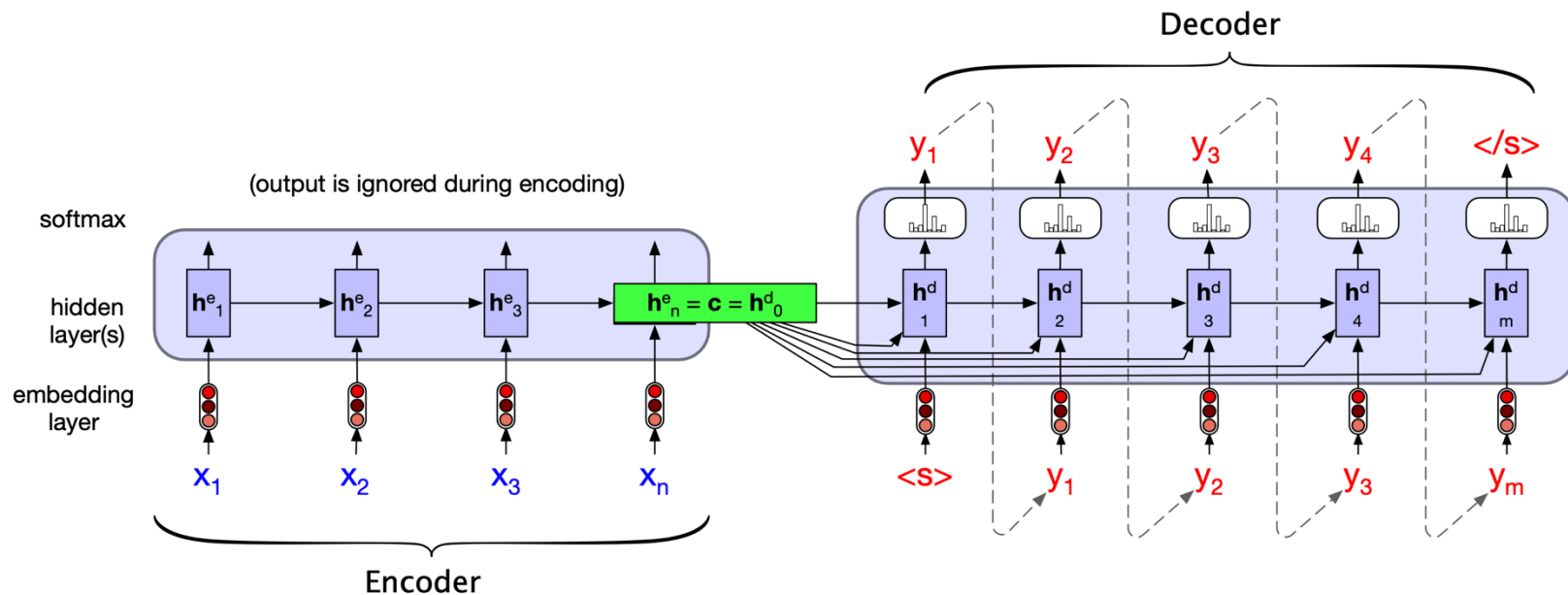
$$\mathbf{h}_t^d = g(\hat{\mathbf{y}}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

output layer (the softmax is
on the output vocabulary)

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{h}_t^d)$$

ENCODER-DECODER - INFERENCE

Example for the case of MT



TRAINING

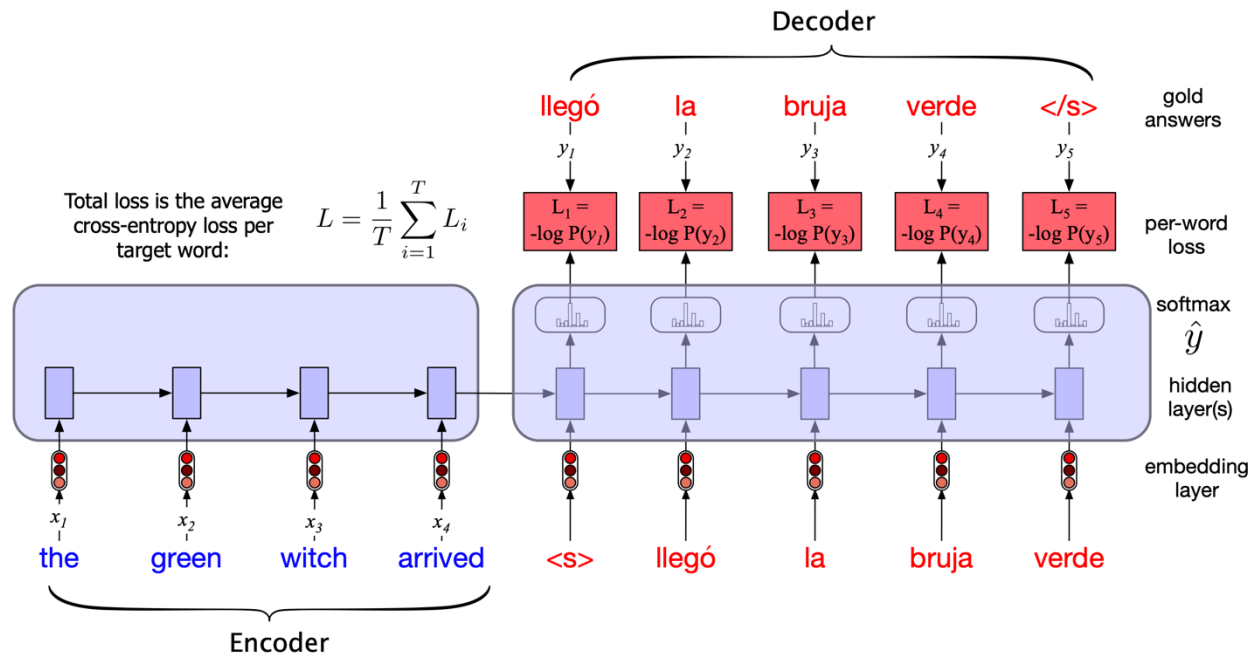
Typically, **teacher forcing** is employed during training (the correct target is provided, rather than the output at the previous step by the decoder).

The network architecture is **trained end-to-end**.

- Loss gradients originating in the decoder are backpropagated all the way through the unfolded architecture, including the encoder.

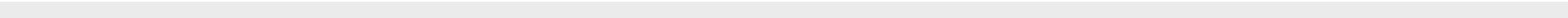
TRAINING

Example for the case of MT





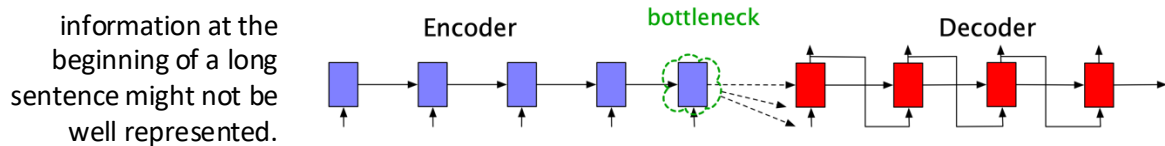
ATTENTION



ENCODER-DECODER BOTTLENECK

So far, the context passed to the decoder is the last state of the last hidden layer in the encoder, i.e., $\mathbf{h}_n^e$.

This represents a computational bottleneck.



it's the only information that the decoder has about the input

it must represent absolutely everything about the meaning of the input sequence.

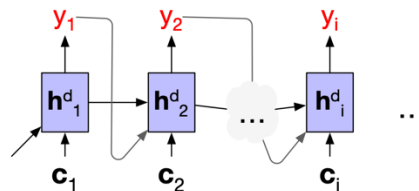
ATTENTION

Goal: allow the decoder to get information from all the hidden states of the encoder.

Idea: the context is a weighted average of all the hidden states of the encoder ... and of the previous decoder state.

$$\mathbf{c}_i = f(\mathbf{h}_1^e \mathbf{h}_2^e \dots \mathbf{h}_n^e, \mathbf{h}_{i-1}^d)$$

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



Each hidden state of the decoder sees a different context

COMPUTING THE CONTEXT

We need to capture (model) the relevance of each encoder state $(\mathbf{h}_1^e \mathbf{h}_2^e \dots \mathbf{h}_n^e)$ to the current decoder state $\mathbf{h}_{i-1}^d$.

A score value: $score(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)$ for each $j = 1, \dots, n$.

DOT PRODUCT ATTENTION

Implements relevance as **similarity**.

Computes the **dot product** between each pair of $\mathbf{h}_{i-1}^d, \mathbf{h}_j^e$

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$

The vector of such scores (varying j) is **normalized with a softmax**, to create a vector of weights, where α_{ij} is relevance of the encoder hidden state $\mathbf{h}_j^e$ for the decoder state $\mathbf{h}_{i-1}^d$:

$$\begin{aligned}\alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)) \\ &= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}\end{aligned}$$

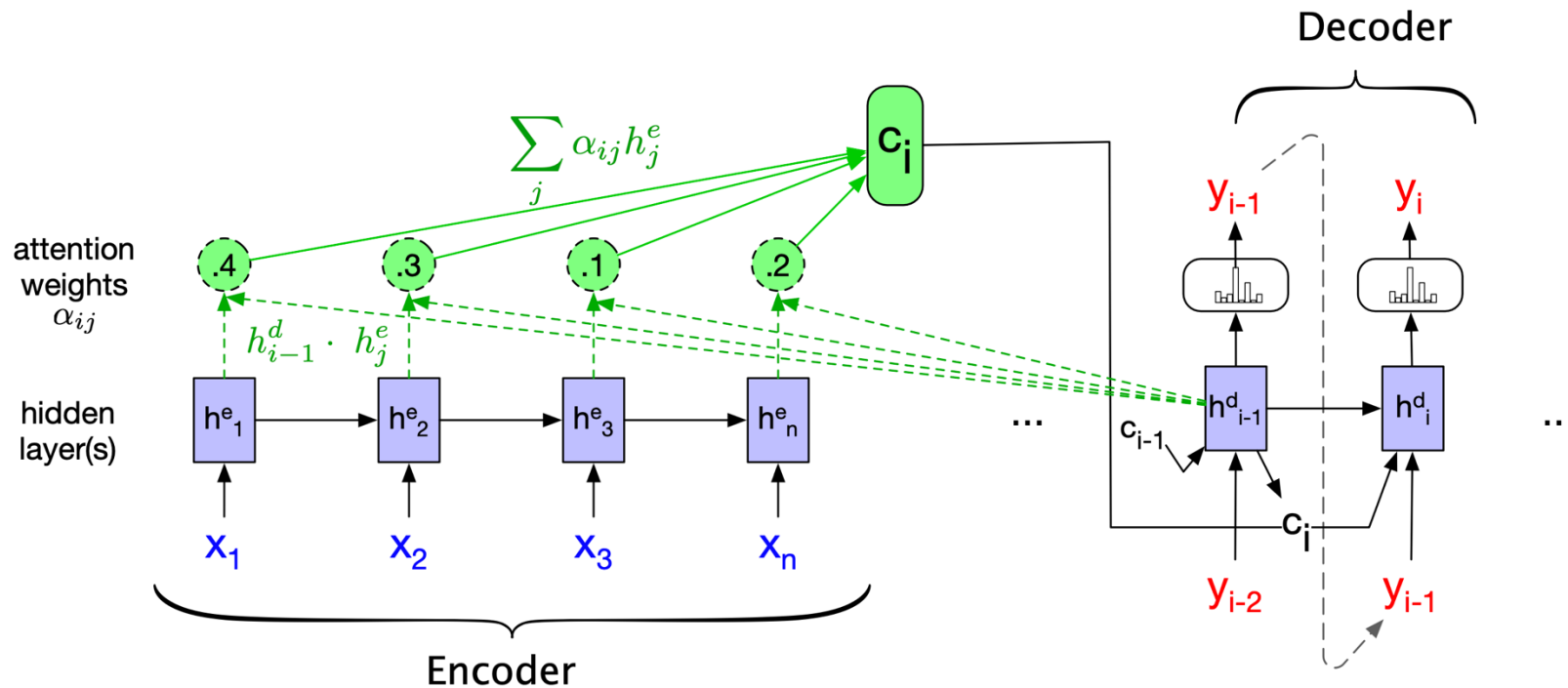
COMPUTING THE CONTEXT

The context at step i is computed as the weighted combination of all the encoder states, weighted by the attention scores.

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$


The context is a fixed-length vector that takes into account information from the entire pool of encoder states.

ENCODER-DECODER WITH ATTENTION



PARAMETRIZED ATTENTION

The score can be parametrized by a specific set of weights.

$$score(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{W}_s \mathbf{h}_j^e$$

Weights in $\mathbf{W}_s$ are **trained end-to-end** and give the flexibility to choose the **aspects of similarity** between the decoder and the encoder states **that are relevant for the specific application**.

Also, allows the encoder and the decoder to have **hidden states with different dimensions**.